

Tecnologie del Linguaggio Naturale — Parte Teorica

Appunti per l'orale — Prof. Daniele Radicioni

Corso di Tecnologie del Linguaggio Naturale — Università di Torino

Indice

1	Semantica Lessicale	2
1.1	Che cos'è la semantica lessicale	2
1.2	Le ontologie	2
1.2.1	Perché le ontologie	2
1.2.2	Cosa si intende per “ontologia”	3
1.2.3	Ontologia e lessico	3
1.3	Design di ontologie	4
1.3.1	DOLCE	4
2	Knowledge Representation	5
2.1	Rappresentazioni strutturate della conoscenza	5
2.2	Reti semantiche	5
2.3	Rappresentazioni gerarchiche ed eredità delle proprietà	6
2.3.1	Il Nixon Diamond	6
2.4	Frame	7
2.5	Teorie del significato	8
2.5.1	Il significato delle parole	8
3	WordNet, FrameNet e Word Sense Disambiguation	11
3.1	WordNet	11
3.1.1	La matrice lessicale	11
3.1.2	Relazioni semantiche	11
3.1.3	I nomi	12
3.1.4	I verbi	12
3.2	Conceptual similarity e misure di similarità in WordNet	13
3.3	Word Sense Disambiguation	13
3.3.1	Feature vector: collocational vs. bag-of-words	14
3.3.2	L'algoritmo di Lesk	14
3.4	FrameNet	15
4	Elaborazione del testo: NLTK e spaCy	17
4.1	I task di base	17
4.2	spaCy	18
5	Modelli di Linguaggio e N-grammi	19
5.1	Perché assegnare una probabilità a una frase	19
5.2	N-grammi	19
5.3	Valutare un modello di linguaggio	20
5.4	Smoothing	20

6	Vector Space Model, Rocchio e Linear Discriminant Analysis	22
6.1	Terminologia e term-document matrix	22
6.2	Cosine similarity	22
6.3	Pesatura dei termini: TF-IDF	23
6.4	Il metodo di Rocchio	23
6.5	Linear Discriminant Analysis	23
7	Word Embeddings: Word2Vec e oltre	25
7.1	Dai vettori sparsi ai vettori densi	25
7.2	Word2Vec e l'algoritmo Skip-gram	26
7.2.1	Il meccanismo di training	26
7.2.2	Skip-gram vs. CBOW	27
7.2.3	Iperparametri principali e limiti	27
7.2.4	Cosa catturano gli embedding: analogie e bias	28
8	Un'introduzione ai Transformer	29
8.1	Dal meaning conflation agli embedding contestualizzati	29
8.2	Architettura	29
8.3	I componenti di ogni strato	30
8.4	Addestramento: pre-training e fine-tuning	31
8.5	Limiti	32

Capitolo 1

Semantica Lessicale

1.1 Che cos'è la semantica lessicale

La **semantica lessicale** è lo studio del significato delle parole e delle relazioni che intercorrono fra loro: si occupa di capire cosa i singoli oggetti lessicali significano, cosa fanno, come possono essere rappresentati e come possono essere combinati per costruire significati più complessi. È il tema portante di questa seconda parte del corso, che affronta in sequenza: il problema della *knowledge representation*, WordNet e FrameNet, i modelli di linguaggio basati su N-grammi, il Vector Space Model e i word/sense embeddings, fino a un'introduzione agli encoder/decoder (Transformer).

Un punto di partenza utile è osservare che le parole sono **flessibili**: un parlante è in grado di ripetere le parole nei sensi già noti, di combinarle in enunciati anche del tutto nuovi, e di estendere una parola o una frase per esprimere sensi che prima non aveva. Queste capacità sono ereditate dagli esseri umani come parte della competenza linguistica, ma non si sviluppano automaticamente in ogni circostanza. È proprio la cooperazione fra queste capacità a rendere possibile la **metalinguistica**: la lingua può essere usata per riflettere su se stessa (“che cosa significa questa parola?”, “si può dire così?”, “si scrive in un modo o nell'altro?”), una funzione riflessiva che manca a sistemi di segni più rigidi.

Questa flessibilità è anche ciò che distingue il linguaggio naturale da linguaggi formali come la matematica o la chimica. Chimica e matematica parlano di aspetti dell'esperienza in modo **rigido**: i simboli hanno un significato fissato univocamente. Le lingue naturali sono invece **deformabili**: il loro significato può essere alterato, esteso, dilatato a seconda del contesto d'uso, ed è proprio questa deformabilità a rendere necessario un intero campo di studio (la semantica lessicale, appunto) invece di un semplice dizionario di corrispondenze forma-significato.

Possibile domanda d'esame. Perché non basta un dizionario per rappresentare il significato delle parole? — Perché il linguaggio naturale è deformabile: il significato di una parola non è fissato una volta per tutte come in un linguaggio formale, ma dipende dal contesto, si estende per polisemia e metafora, e le relazioni fra parole (sinonimia, iponimia, ecc.) sono spesso questione di grado più che di identità netta. Un dizionario elenca le forme; la semantica lessicale studia la struttura del significato e le relazioni sistematiche fra i concetti lessicalizzati.

1.2 Le ontologie

1.2.1 Perché le ontologie

Per affrontare lo studio sistematico del significato, il corso adotta il punto di vista delle **ontologie**, per due ragioni complementari:

- una ragione **metodologica**: le ontologie permettono di adottare un piano interdisciplinare, a un livello di generalità sufficientemente alto da poter parlare di significato indipendentemente da una particolare applicazione;
- una ragione **architetturale**: un'ontologia può giocare un ruolo centrale all'interno di un sistema informativo, fungendo da struttura di riferimento condivisa fra componenti diverse.

1.2.2 Cosa si intende per “ontologia”

Il termine *ontologia* viene ripreso dalla filosofia ma acquisisce, in questo contesto, un significato più operativo. In **filosofia**, ontologia indica lo studio dell'essere e delle sue categorie fondamentali: un'ontologia filosofica definisce un insieme di primitive rappresentazionali con cui modellare un dominio di conoscenza o di discorso, organizzato in categorie e relazioni, le cui definizioni includono informazioni sul significato e vincoli su come applicarle in modo consistente (una categoria, per esempio, rappresenta tutti i tipi di entità che possono fungere da soggetto in un predicato). Il problema di questa definizione è che è così generale da ammettere che quasi qualsiasi cosa possa essere considerata un'ontologia.

Una seconda definizione, più utile in pratica, vede l'**ontologia come concettualizzazione**: un'ontologia è una specifica esplicita di una concettualizzazione, cioè una struttura formale di un pezzo della realtà così come è percepito e organizzato da un agente, indipendentemente dal vocabolario utilizzato o dall'occorrenza in una situazione specifica. Una conseguenza interessante di questa definizione è che situazioni diverse, che coinvolgono lo stesso oggetto ma sono descritte con vocabolari differenti, possono in realtà condividere la stessa concettualizzazione sottostante.

È utile distinguere *ontologia* da *semantica*: un'ontologia riguarda ciò che esiste (l'inventario di categorie ed entità), mentre la semantica si riferisce a come il linguaggio esprime ciò che esiste. Aspetti diversi del linguaggio hanno ruoli diversi rispetto all'ontologia. Coerentemente, si distinguono due componenti di una *knowledge base*: una componente **terminologica** (l'ontologia vera e propria), indipendente da un particolare stato del mondo e pensata per supportare servizi terminologici; e una componente **asserzionale**, che riflette invece stati specifici ed è pensata per il problem solving.

1.2.3 Ontologia e lessico

A livello lessicale, il **lessico** di una lingua è l'elenco delle sue parole, comprensivo del vocabolario e della conoscenza su come le parole vengono usate. Fra le parole si stabiliscono **relazioni lessicali** che ritroveremo sistematicamente in WordNet: la *sinonimia* (stesso significato), l'*iponimia* (relazione di sottoclasse) e la sua inversa *iperonimia* (relazione di superclasse), la *meronimia* (essere costituente di qualcosa) e la sua inversa *olonimia* (essere il tutto), l'*antonimia* (significato opposto).

Il lessico, tuttavia, ha caratteristiche che lo distinguono da una tassonomia ontologica pulita:

- **sovrapposizione dei sensi** (*overlapping word senses*): mentre in un'ontologia le sottocategorie di una categoria generale sono pensate come mutuamente esclusive, nel lessico esistono sovrapposizioni di significato, per esempio fra quasi-sinonimi;
- **buchi lessicali**: il lessico di una lingua può omettere di lessicalizzare (cioè di dare un nome a una sola parola) categorie ontologiche che pure esistono concettualmente; queste categorie richiedono perifrasi, cioè giri di parole, per essere espresse;
- **lessici tecnici**: nei contesti tecnici il linguaggio è molto più vicino all'ontologia del dominio, perché i termini tecnici sono definiti con l'esplicito obiettivo di corrispondere uno a uno alle categorie del dominio.

Le ontologie, in definitiva, si costruiscono per condividere la comprensione delle entità di un certo dominio fra persone o sistemi diversi, per permettere il riutilizzo di dati e informazioni già strutturati, e per creare e sostenere comunità di ricercatori che lavorano su basi concettuali comuni.

1.3 Design di ontologie

Un passo preliminare nel progettare un'ontologia è distinguere due grandi categorie di “cose” rappresentabili: le **entità**, oggetti che continuano a esistere per un periodo di tempo mantenendo la propria identità, e gli **eventi**, oggetti che accadono, si svolgono o si sviluppano nel tempo. Un'**ontologia fondazionale** è un'ontologia che cattura un insieme di distinzioni di base, valide trasversalmente in domini diversi; esempi celebri di ontologie fondazionali sono DOLCE, SUMO e CYC.

1.3.1 DOLCE

DOLCE (Descriptive Ontology for Linguistic and Cognitive Engineering) è un'ontologia fondazionale il cui scopo è rappresentare le strutture concettuali di base che emergono dal linguaggio naturale e dalla cognizione umana, piuttosto che da considerazioni puramente metafisiche. Le scelte alla base di DOLCE sono:

- gli **endurant**: entità che sono completamente presenti in ogni momento della loro esistenza (per esempio un oggetto fisico: in ogni istante in cui esiste, esiste per intero);
- i **perdurant**: eventi che si estendono nel tempo e sono presenti solo *parzialmente* in ogni istante (per esempio una corsa: nell'istante t è avvenuta solo una parte della corsa, non la corsa intera);
- le **quality**: proprietà specifiche di un'entità, che dipendono da essa (per esempio il colore di una rosa);
- un **approccio moltiplicativo**: DOLCE ammette che oggetti ed eventi differenti possano essere co-localizzati nello stesso spazio-tempo (per esempio, la statua e il pezzo di argilla di cui è fatta occupano lo stesso spazio, ma sono considerate entità concettualmente distinte).

Per decidere se un'entità è sottoclasse di un'altra, DOLCE richiede che le due condividano gli stessi **criteri di identità**: proprietà necessarie che permettono di stabilire se due entità sono davvero in relazione di sottoclasse. Per esempio, “intervallo di tempo” non può essere sottoclasse di “durata di tempo” perché i due concetti si affidano a criteri di identità differenti (un intervallo ha estremi definiti, una durata è solo una misura di lunghezza temporale).

La distinzione fra *endurant* e *perdurant* ha conseguenze concrete: gli *endurant* possono *cambiare* nel tempo (lo stesso *endurant* può avere proprietà incompatibili in istanti diversi: una mela verde può diventare rossa), mentre i *perdurant* *non* cambiano, in quanto hanno già una collocazione spazio-temporale ben definita dagli *endurant* che vi partecipano (una corsa non “cambia”, semplicemente si estende nel tempo). La relazione che collega i due tipi di entità è la **partecipazione**: un *endurant* vive partecipando a qualche *perdurant* (una persona esiste come *endurant* partecipando all'evento-*perdurant* della sua corsa).

Le *qualities* sono infine caratteristiche di base che possono essere percepite e misurate (forma, colore, suono, ecc.), specifiche per ogni individuo; è importante distinguere fra una qualità e il suo *valore* (per esempio, il colore di una rosa specifica è una qualità individuale, ma la particolare sfumatura di rosso che assume è il suo valore in una regione di qualità).

Possibile domanda d'esame. Qual è la differenza fra *endurant* e *perdurant* in DOLCE? — Un *endurant* è completamente presente in ogni istante della sua esistenza e può cambiare proprietà nel tempo (es. un oggetto fisico); un *perdurant* si estende nel tempo ed è presente solo parzialmente in ogni istante, e non cambia perché la sua collocazione spazio-temporale è già determinata dagli *endurant* che vi partecipano (es. un evento). La relazione fra i due è la partecipazione: gli *endurant* partecipano ai *perdurant*.

Capitolo 2

Knowledge Representation

2.1 Rappresentazioni strutturate della conoscenza

Durante la prima fase dell'intelligenza artificiale l'attenzione era concentrata sullo sviluppo di metodi di risoluzione dei problemi del tutto generali, indipendenti dal dominio applicativo. A partire dalla seconda metà degli anni '60 ci si rese conto che, per affrontare problemi reali (e non più solo domini astratti e semplificati), era necessaria una conoscenza specifica sul mondo in cui il sistema doveva operare: nasce così il problema della **knowledge representation**, cioè trovare formalismi adeguati a rappresentare le conoscenze necessarie.

Un sistema di conoscenza, in questa prospettiva, deve sempre comprendere due componenti: un **linguaggio di rappresentazione**, cioè un insieme di strutture sintattiche adatte a codificare le informazioni, e un insieme di **regole o operazioni** per manipolare quel linguaggio. Non basta rappresentare l'informazione: l'implementazione delle regole deve portare alle inferenze desiderate, e le regole stesse devono poter essere espresse come procedure eseguibili.

Una prima possibilità per rappresentare la conoscenza è usare formule di logica proposizionale o del primo ordine, ciascuna rappresentante una proposizione indipendente. Il problema di questo approccio puramente logico è che non è possibile collegare fra loro le varie formule organizzandole in blocchi omogenei: la logica, da sola, non basta, e richiede ulteriori meccanismi computazionali per aggregare conoscenze elementari in strutture più complesse, in cui le conoscenze relative a uno stesso oggetto siano direttamente accessibili in un unico punto della struttura.

2.2 Reti semantiche

Le **reti semantiche** sono un formalismo nato dai primi progetti di traduzione automatica. La loro caratteristica comune è di utilizzare una struttura a grafo, in cui i nodi rappresentano concetti e gli archi rappresentano relazioni fra concetti (o proprietà dei concetti). La versione più semplice di rete semantica è il **grafo relazionale**: una struttura in cui ogni nodo rappresenta un'entità, e ogni arco, etichettato con il nome della relazione, la collega ad altre entità. Una relazione particolarmente importante, che ricorrerà costantemente, è **isA**: esprime il tipo di concetto che un nodo rappresenta (per esempio, collegare "elefante" a "mammifero" con un arco isA permette di ereditare per transitività tutte le proprietà dei mammiferi anche agli elefanti).

Un grafo relazionale implementa, di fatto, un sottoinsieme del calcolo dei predicati del primo ordine: gli archi corrispondono ai predicati e i nodi ai termini. Questo formalismo ha però dei limiti espressivi evidenti: la congiunzione è rappresentata solo implicitamente (mettendo insieme più archi), mentre risulta difficile rappresentare la disgiunzione, l'implicazione, o la quantificazione universale. Inoltre le relazioni espresse dagli archi sono per natura binarie, mentre i predicati logici possono avere qualsiasi arietà. Una possibile soluzione è tradurre ogni relazione in una serie di relazioni binarie: questo accresce la granularità (e quindi l'espressività) della rappresentazione, ma richiede di introdurre nodi aggiuntivi per rappresentare oggetti, insiemi ed eventi, e fa sì che

i predicati con arietà superiore a due si “esplodano” in più relazioni binarie (una che chiarisce il tipo di predicato, le altre che ne esplicitano ruolo e funzione degli argomenti).

Un'estensione più espressiva sono le **reti proposizionali**, i cui nodi possono rappresentare non solo entità semplici ma intere proposizioni. Ammettere nodi proposizionali accresce di molto l'espressività del linguaggio (sono state proposte reti fortemente legate alla logica del primo ordine), e permette in particolare di introdurre la **negazione**: un arco può collegare il risultato della negazione con la proposizione negata, distinguendo fra negazione di un'intera proposizione e negazione di una proposizione incassata dentro un'altra. La **disgiunzione** viene introdotta subito dopo la negazione, perché la congiunzione è già implicita nella struttura a grafo e, grazie alle leggi di De Morgan, disgiunzione e congiunzione possono essere espresse l'una in funzione dell'altra una volta disponibile anche la negazione. In generale, la scelta di quale tipo di rete utilizzare va fatta bilanciando leggibilità, flessibilità, efficienza e facilità di espressione.

2.3 Rappresentazioni gerarchiche ed eredità delle proprietà

Molta della conoscenza che vogliamo rappresentare è naturalmente organizzata in **gerarchie**: entità raggruppate in classi, a loro volta raggruppate in sottoclassi. Queste gerarchie non riguardano solo oggetti, ma si estendono ad azioni, eventi, stati, proprietà. Il meccanismo di **eredità delle proprietà** afferma che le proprietà asserite per un nodo valgono automaticamente anche per i nodi a un livello inferiore della gerarchia: se la rete isA è organizzata ad albero, per sapere se un concetto C gode di una proprietà P basta risalire i suoi antenati e verificare se qualcuno di essi possiede P , un procedimento più efficiente di un processo di inferenza generico.

Questo meccanismo porta due vantaggi concreti: un'**economia di rappresentazione** (una proprietà condivisa da tutta una sottogerarchia viene asserita una sola volta, al livello più alto in cui si applica, invece di essere ripetuta su ogni nodo), e una **semplificazione della manutenzione** (modificare quella proprietà richiede una sola operazione). Il rovescio della medaglia è il trattamento delle eccezioni: gli uccelli di solito volano, ma non tutti (il pinguino no); i mammiferi di solito partoriscono i figli, ma l'ornitorinco no. Il meccanismo standard per gestire le eccezioni è la **validità per default**: si rappresentano conoscenze che valgono fino a prova contraria, e le eccezioni vengono registrate direttamente in corrispondenza dei nodi a cui si applicano, in modo che vengano incontrate (e quindi prevalgano) prima di raggiungere il valore di default ereditato dall'alto.

Le cose si complicano quando una classe ha più di una classe sovraordinata (cioè $C \text{ isA } A$ e $C \text{ isA } B$ con $A \neq B$): la rete semantica smette di essere un albero e diventa un grafo, e si parla di **ereditarietà multipla**. Ammettere l'ereditarietà multipla ha un costo: il tempo di ricerca passa da lineare a esponenziale (e non esistono euristiche generali per contenerlo), e soprattutto, in caso di conflitto fra i valori ereditati da genitori diversi, non esiste un criterio risolutivo che abbia carattere generale — tutto dipende dall'algoritmo di ricerca scelto (ricerca in profondità con restituzione del primo valore trovato, oppure ricerca in ampiezza, cioè *shortest path inheritance*).

2.3.1 Il Nixon Diamond

L'esempio canonico di questo problema è il cosiddetto **Nixon Diamond**: Richard Nixon era contemporaneamente quacchero (e i quaccheri sono tipicamente pacifisti) e repubblicano (e i repubblicani dell'epoca erano tipicamente “falchi”, cioè non pacifisti). Un algoritmo che restituisse tutti i valori ereditabili concluderebbe che Nixon è sia pacifista sia non-pacifista: un risultato palesemente inconsistente. Un algoritmo di ricerca in profondità restituirebbe un valore unico ma dipendente arbitrariamente dall'ordine in cui vengono esaminati i nodi figli; la ricerca in ampiezza sceglierebbe il cammino più corto fra Nixon e la proprietà cercata, il che nell'esempio classico porta a concludere che Nixon è pacifista solo perché quel cammino risulta più breve — una scelta tecnica, non un giudizio informato sul mondo.

Una via d'uscita concettuale è ammettere che la rete stessa sia **ambigua**, cioè in grado di esprimere più interpretazioni contemporaneamente: ciascuna interpretazione, presa singolarmente, è consistente, e le due interpretazioni diventano inconsistenti solo se considerate congiuntamente. Si parla in questo caso di **dissonanza cognitiva**: in quanto quacchero Nixon è pacifista, in quanto repubblicano non lo è, e quale aspetto prevalga dipende da quale dei due tratti giudichiamo più rilevante nel contesto.

Possibile domanda d'esame. Cos'è il Nixon Diamond e perché è un problema per l'ereditarietà multipla? — È il caso in cui un nodo (Nixon) eredita da due classi sovraordinate (quacchero, repubblicano) proprietà incompatibili riguardo al pacifismo. Mostra che, in presenza di ereditarietà multipla, non esiste un criterio generale per risolvere i conflitti fra valori ereditati: qualunque strategia di ricerca (profondità, ampiezza) fornisce un risultato che dipende da dettagli implementativi (ordine di visita, lunghezza del cammino) piuttosto che da un ragionamento sul dominio. Una possibile via d'uscita è considerare la rete come ambigua e parlare di dissonanza cognitiva.

Le reti semantiche hanno anche un altro problema strutturale: non esiste distinzione fra nodi che rappresentano *individui* e nodi che rappresentano *classi o insiemi di individui*. Il legame isA viene quindi usato indifferentemente per denotare sia l'**appartenenza** (un elemento nel suo insieme) sia l'**inclusione** (un insieme in un insieme più ampio); di conseguenza non si riesce a distinguere fra proprietà vere per tutti gli individui di una classe e proprietà vere della classe *in quanto tale*. L'esempio classico: “gli elefanti sono numerosi” è vero della specie nel suo complesso, non di nessun elefante individualmente (nessun singolo elefante è “numeroso”). In origine, inoltre, mancava alle reti semantiche una semantica formale condivisa: il significato di una rete dipendeva soltanto dalla natura delle procedure che la manipolavano, rendendo impossibile separare la semantica di una rete dal suo uso concreto.

2.4 Frame

I **frame**, introdotti da Minsky nel 1975, sono un formalismo che condivide diversi punti di contatto con le reti semantiche, ma parte da un'osservazione diversa: le persone, di fronte a una situazione nuova, non ragionano da zero. Recuperano invece dalla memoria una rappresentazione a carattere generale — il frame — che viene poi raffinata e adattata per rendere conto dei dettagli della situazione specifica. Un frame è quindi una struttura che rappresenta le conoscenze generali che un individuo ha riguardo a situazioni, luoghi, oggetti, personaggi stereotipati, e fornisce una cornice concettuale all'interno della quale i nuovi dati vengono interpretati alla luce dell'esperienza passata.

L'esempio classico è il *frame del ristorante*: sedendoci per la prima volta in un ristorante che non conosciamo non ci sentiamo del tutto spaesati, perché ci aspettiamo di vedere dei tavoli, che un cameriere si avvicini per prendere le ordinazioni, e così via. È proprio questo che rende i frame utili: un sistema che li utilizza può formulare previsioni e avere aspettative (se aprendo la porta del ristorante ci venisse incontro un palombaro in tuta da sub, la sorpresa deriverebbe esattamente dalla violazione del frame attivato). I frame aiutano anche a interpretare situazioni ambigue sfruttando il contesto: un oggetto metallico con le impugnature divaricate, anche se coperto da un tovagliolo, viene riconosciuto come uno schiaccianoci se siamo a tavola; lo stesso oggetto, in un'officina meccanica, verrebbe più naturalmente interpretato come una pinza. In generale, i frame organizzano la conoscenza di un dominio in modo da facilitare sia il reperimento delle informazioni sia i processi inferenziali necessari per agire in modo intelligente.

Una struttura di conoscenza di livello ancora più alto, che integra e organizza intere sequenze di frasi, è lo **script** (o copione). Si confrontino queste due sequenze: (A) “Giacomo andò al ristorante. Chiese al cameriere una bistecca con patatine. Pagò il conto e uscì”; (B) “Giacomo andò al parco. Chiese al nano un topolino. Prese la scatola e se ne andò”. Le due sequenze

sono paragonabili per struttura sintattica e complessità semantica, ma solo la prima ha un senso pienamente compiuto per un lettore, perché esiste uno script condiviso (“andare al ristorante”) che permette di collegare e dare senso alle singole frasi; per la seconda sequenza manca uno script altrettanto condiviso, e il testo risulta spiazzante o incomprensibile.

Dal punto di vista strutturale, un frame ha un nome univoco, e le sue caratteristiche sono rappresentate da un insieme di **slot** (caselle in cui va inserito un certo tipo di informazione), ciascuno eventualmente dotato di un **valore di default**. Gli elementi di un frame sono collegati da relazioni *isA* o *ako* (“a kind of”), che permettono l’ereditarietà: le proprietà ai livelli alti della gerarchia sono fisse, mentre ai livelli più bassi (sottoclassi o istanze) possono essere specializzate, con eventuali conflitti in caso di ereditarietà multipla (esattamente come nelle reti semantiche). Uno slot può a sua volta contenere una struttura complessa, e per rendere efficiente il calcolo dei suoi valori si usano procedure come *if-needed* (il valore viene calcolato solo quando serve) e *if-added* (una procedura scatta solo quando si tenta di riempire quello slot). A differenza delle reti semantiche pure, i frame rappresentano la conoscenza in modo dichiarativo ma senza offrire di per sé una semantica formale: si presuppone semplicemente l’esistenza di procedure esterne in grado di utilizzare le informazioni contenute negli slot.

Riguardo ai concetti organizzati nei frame, si distinguono tre livelli: il **livello di base**, che costituisce il modo più naturale di categorizzare oggetti ed entità; il **livello superordinato/subordinato**, rispettivamente generalizzazioni e specializzazioni del livello di base; e i **prototipi**, cioè i membri più tipici di una categoria, rispetto ai quali l’appartenenza di un nuovo elemento viene giudicata per maggiore o minore somiglianza (un passero è un rappresentante migliore della categoria “uccello” rispetto a un airone, che a sua volta è un rappresentante migliore rispetto a uno struzzo).

2.5 Teorie del significato

Come si rappresenta, in generale, il significato di un concetto? Il corso presenta tre teorie che si possono considerare complementari più che alternative: la **teoria dei prototipi**, appena vista, per cui un concetto è rappresentato da un suo membro tipico (o da una combinazione di tratti tipici) rispetto al quale si giudicano per somiglianza i nuovi casi; la **teoria degli esempi**, per cui la rappresentazione mentale di un concetto è l’insieme di alcuni esempi concreti già incontrati di quella categoria (non un prototipo astratto, ma casi specifici memorizzati); e la **teoria**, per cui i concetti fanno parte di una comprensione più ampia e strutturata del significato, collegata sistematicamente ad altri concetti (per esempio, sapere cos’è un “uccello” implica anche una teoria ingenua di biologia che lo collega ad altri concetti come “volare”, “nido”, “uovo”).

Un ulteriore modo di classificare le abilità concettuali si ispira alle **teorie del doppio processo** (*dual process theories*) nate in psicologia del ragionamento (Kahneman, *Thinking, Fast and Slow*, 2011, fra gli altri): esisterebbero due tipi di processi cognitivi, il **Sistema 1**, implicito, inconscio, automatico, evolutivamente precoce, parallelo e veloce, pragmatico e contestualizzato; e il **Sistema 2**, esplicito, cosciente, controllabile, evolutivamente tardivo, legato al linguaggio, sequenziale e lento, logico e astratto. Applicata alla rappresentazione concettuale, questa distinzione si traduce nel contrasto fra una **categorizzazione non monotona** basata sulla conoscenza tipica (Sistema 1: veloce ma soggetta a eccezioni, come nell’eredità delle proprietà con default) e una **categorizzazione monotona** basata su processi deliberativi, sequenziali (Sistema 2: più lenta ma logicamente coerente).

2.5.1 Il significato delle parole

Occupandoci di parole singole, due problemi si presentano immediatamente: la **polisemia** (una parola può avere più di un significato) e la **semantica frasale** (il modo in cui i significati delle singole parole si combinano in un enunciato). Centrale in entrambi i casi è la nozione di **contesto**: l’insieme degli elementi adiacenti a una parola, che può essere sintattico (le proprietà sintattiche

degli elementi vicini), semantico (le loro proprietà di significato), linguistico in generale, oppure situazionale/pragmatico (richiede conoscenza del mondo esterna alla lingua stessa).

L'**ambiguità** è la proprietà di una forma lessicale di avere più di un significato, e si distingue in **contrastativa** (o omonimia: significati fra loro scollegati, come “riva” del fiume e “riva” come voce del verbo arrivare) e **complementare** (o polisemia vera e propria: sensi correlati che emergono in contesti diversi, come “giornale” inteso come oggetto fisico o come istituzione). Il fatto che le lingue tollerino così tanta ambiguità si spiega con un **principio di economicità linguistica**: riusare le stesse forme per più significati contiene le dimensioni del lessico che un parlante deve imparare e ricordare. In generale, i verbi tendono a essere più polisemici dei sostantivi.

Diverse **teorie del significato delle parole** si sono succedute. La **teoria referenziale** vede le parole come strumenti con cui ci si riferisce a ciò che esiste: il significato di una parola sta nella sua capacità di stabilire una relazione con elementi della realtà extralinguistica, tramite *denotazione* (indicare la classe di elementi) o *designazione* (indicare un elemento particolare della classe). La **teoria mentalista** (o concettuale) arricchisce quella referenziale ipotizzando che il riferimento fra parole e realtà non sia diretto, ma mediato dall'immagine mentale dei concetti nella mente del parlante: questo permette di parlare non solo di cose esistenti, ma anche di entità astratte, immaginarie o ipotetiche, spostando l'accento sugli aspetti psicologici della concettualizzazione. I **concetti cognitivi** legati a questa mediazione sono entità instabili, che possono differire da individuo a individuo e da cultura a cultura; i **concetti lessicalizzati** sono invece più stabili, condivisi a livello sociale, ma variano da lingua a lingua.

La **teoria strutturale** sposta ulteriormente la prospettiva: il significato di un termine non sta nel suo riferirsi a un oggetto, ma nel valore che quel termine assume in relazione a tutte le altre parole dello stesso campo semantico (il suo contenuto informativo è quindi relazionale, non referenziale). Infine, la **teoria distribuzionale** — tornata di grande attualità con la disponibilità di enormi corpora testuali — sostiene che il significato di una parola è determinato in larga misura dall'insieme delle altre parole con cui essa co-occorre: è la base concettuale diretta delle rappresentazioni vettoriali che vedremo più avanti (VSM, word embeddings), riassunta efficacemente dalla **metafora geometrica del significato**: i significati delle parole corrispondono a punti in uno spazio multidimensionale, tali che punti vicini corrispondano a parole di significato affine — l'idea che sta alla base della cosine similarity.

Possibile domanda d'esame. Che relazione c'è fra la teoria distribuzionale del significato e i word embeddings che vedremo più avanti nel corso? — La teoria distribuzionale sostiene che il significato di una parola sia determinato dall'insieme delle parole con cui essa co-occorre (Harris, Firth: “a word is characterized by the company it keeps”). Questa è esattamente l'ipotesi computazionale alla base sia del Vector Space Model sia di word2vec: rappresentare ogni parola con un vettore costruito a partire dai suoi contesti d'uso, in modo che parole distribuzionalmente simili risultino vicine nello spazio vettoriale (metafora geometrica del significato), misurabile con la cosine similarity.

Un ultimo nodo concettuale riguarda il **principio di composizionalità del significato**: spiega come il significato di un intero enunciato si formi a partire dal significato dei suoi elementi lessicali. Questo principio, tuttavia, viene sistematicamente meno in almeno tre casi: le espressioni idiomatiche (“prendere in giro” non è la somma di “prendere” e “giro”), gli usi metaforici, e la polisemia. Per affrontare questi casi si possono seguire due approcci complementari: l'**enumerazione dei sensi** (i diversi sensi di una parola sono elencati esplicitamente, insieme ai contesti che li attivano) oppure una **concezione dinamica**, per cui le parole sono entità permeabili e il significato di ciascuna interagisce con quello delle parole adiacenti. In quest'ultima prospettiva rientrano alcuni meccanismi di interazione semantica: la **co-composizione** (il significato di un verbo ha una parte di base fissa, ma viene ridefinito e specializzato dal suo complemento: non è lo stesso “cuocere” in “cuocere la pasta” e in “cuocere al sole”), la **forzatura di tipo** (*type coercion*: un verbo, combinato con un nome specifico, lo spinge a significare qualcosa anche variandone il tipo

semantico atteso), e il **legamento selettivo** (un aggettivo può selezionare solo una porzione specifica del significato del nome che modifica, per esempio “veloce” applicato a “computer” seleziona la velocità di calcolo, non le dimensioni fisiche).

Capitolo 3

WordNet, FrameNet e Word Sense Disambiguation

3.1 WordNet

WordNet è un sistema di riferimento lessicale online per l'inglese, il cui design è esplicitamente ispirato dalle teorie psicolinguistiche sulla memoria lessicale umana. Nasce nel 1985 a Princeton, per iniziativa di un gruppo di psicologi e linguisti che intendevano fornire un aiuto alla ricerca nei dizionari a livello *concettuale*, pensato per essere usato in congiunzione con un dizionario online convenzionale, non per sostituirlo.

Il punto di partenza è una critica ai dizionari “classici”: in un dizionario alfabetico, parole con significato simile o collegato finiscono per essere disperse lontano l'una dall'altra (perché vicine sono solo le parole che iniziano con le stesse lettere), e non esiste un modo naturale per il lettore di navigare fra concetti imparentati. Trovare una parola in mezzo a migliaia di voci è banale per un computer; sarebbe uno spreco limitarsi a questo, quando si può invece organizzare il lessico attorno al significato.

WordNet divide il lessico in quattro categorie sintattiche, ciascuna organizzata secondo principi diversi: i **nomi** (organizzati gerarchicamente), i **verbi** (organizzati come un insieme di relazioni non gerarchiche), gli **aggettivi** e gli **avverbi**.

3.1.1 La matrice lessicale

Per evitare ambiguità terminologiche, WordNet distingue fra *word form* (l'espressione fisica di una parola) e *word meaning* (il concetto lessicalizzato espresso da quella forma). Si immagini una **matrice lessicale** in cui le colonne rappresentano le word form e le righe i word meaning: se due voci (entry) condividono la stessa colonna, le due parole sono **polisemiche** (stessa forma, significati diversi); se due voci condividono la stessa riga, le due parole sono **sinonime** (forme diverse, stesso significato). Un **synset** è precisamente l'insieme delle word form che, in un dato contesto, possono esprimere lo stesso word meaning: il significato di una parola può essere comunicato scegliendo una qualunque forma che lo esprima all'interno del synset, sul presupposto che chi conosce l'inglese abbia già assimilato il concetto e sia in grado di riconoscerlo a partire da una qualsiasi delle parole del synset. Quando un synset non ha sinonimi a sufficienza per essere autoesplicativo, la sua definizione informale (**gloss**) funge essa stessa da chiarimento del senso.

3.1.2 Relazioni semantiche

WordNet è organizzato attorno a **relazioni semantiche**: puntatori fra synset (quindi fra significati, non fra singole parole), tipicamente simmetriche. Le principali sono: la **sinonimia**, relazione lessicale di similarità di significato (in teoria, due espressioni sono sinonime se sostituendo l'una con l'altra il valore di verità della frase non cambia; in pratica i sinonimi perfetti sono rarissimi,

e si parla sempre di sinonimia relativa a un contesto linguistico specifico); l'**antonimia**, relazione lessicale — non semantica — fra le *forme* delle parole (“ricco” è antonimo di “povero”, ma i due concetti non sono opposti nello stesso senso in cui lo sono, per esempio, “ascesa” e “caduta”, che sono opposti concettualmente ma non sono considerati antonimi in senso lessicale); l'**iponimia**, relazione *ako* (a kind of), transitiva e asimmetrica; la **meronimia**, relazione *hasa* (part-whole), anch'essa transitiva e asimmetrica (in modo limitato).

3.1.3 I nomi

I nomi sono organizzati in gerarchie di iponimia/iperonimia: una definizione fornisce un termine sovraordinato più alcune caratteristiche distintive. Queste definizioni hanno dei limiti dichiarati: non specificano quale, fra i vari sensi del termine sovraordinato, sia quello corretto; non forniscono informazioni sui termini coordinati (cioè gli altri iponimi dello stesso iperonimo); puntano sempre “verso l'alto” della gerarchia, mai lateralmente o verso il basso. Un problema classico dei dizionari sono le **definizioni circolari** (*A* usato per definire *B* e *B* per definire *A*): WordNet impone una struttura ad albero proprio per evitare la circolarità.

La scelta di organizzare i nomi come un sistema di ereditarietà non è casuale, ma riflette un'**assunzione psicolinguistica**: esiste evidenza sperimentale che una struttura gerarchica sia più facile da concettualizzare per la mente umana. L'esperimento classico (Collins e Quillian) misura i **tempi di reazione**: le persone rispondono più velocemente all'affermazione “un canarino può cantare” che a “un canarino può volare”, perché la prima proprietà è associata direttamente al nodo “canarino”, mentre la seconda va recuperata risalendo la gerarchia fino a “uccello” — un passaggio che richiede tempo aggiuntivo per estrarre la caratteristica dal concetto sovraordinato. Questo risultato resta comunque aperto a un dibattito interpretativo: non è chiaro se l'informazione generica sia davvero ereditata dinamicamente oppure semplicemente “salvata” in modo ridondante a ogni nodo, dato che concetti collegati dallo stesso legame semantico vengono confermati con tempi di reazione diversi fra loro; una possibile via di uscita è ritenere che l'assunzione di ereditarietà resti corretta, ma che il tempo di reazione misuri piuttosto una distanza *pragmatica* (quanto un'associazione è frequente/tipica) più che una distanza puramente strutturale.

In pratica, in WordNet i nomi sono divisi in un piccolo numero di insiemi semantici (25 “punti iniziali”, ciascuno radice di una propria gerarchia, con ereditarietà multipla ammessa). Esiste tipicamente un **livello di base** (*basic level*) a cui sono attaccate la maggior parte delle caratteristiche distintive: sopra questo livello i concetti sono brevi e generali, sotto sono via via più specifici. Le caratteristiche distintive dei concetti sono di tre tipi: **attributi** (espressi dagli aggettivi che possono normalmente modificare quel nome), **parti** (relazione di meronimia: se *x* è meronimo di *y* allora *y* è olonimo di *x*; i meronimi sono ereditati dagli iponimi in modo asimmetrico e transitivo, seppur limitatamente), e **funzioni** (descrizioni di ciò che le istanze di un concetto fanno normalmente, tipicamente espresse da verbi: un oggetto che non svolge bene la funzione tipica di una categoria difficilmente viene considerato un buon membro di quella categoria).

3.1.4 I verbi

I verbi forniscono il framework relazionale e semantico delle frasi: la loro **sottocategorizzazione** specifica quanti argomenti un verbo richiede e di che tipo, ciascun argomento ricopre un ruolo semantico (una funzione rispetto al significato dell'azione), e le *selectional restriction* specificano le proprietà semantiche ammesse per la classe di nomi che riempie ciascun ruolo — tutta informazione che fa parte della voce lessicale del verbo nel dizionario mentale.

Rispetto ai nomi, i verbi in WordNet hanno alcune peculiarità: sono molto meno numerosi, ma generalmente più polisemici (i verbi più comuni sono estremamente polisemici); sono più flessibili e possono cambiare significato a seconda dei nomi con cui co-occorrono. Soprattutto, non vale per i verbi una vera relazione di iponimia (non si può dire in senso proprio che un verbo v_1 sia “a kind of” v_2), mentre è presente un concetto di **intensità/modo**, chiamato **troponimia**: un verbo v_1 sta in relazione con un verbo v_2 eseguendo l'azione di v_2 in un modo particolare (per esempio,

“sussurrare” è un troponimo di “parlare”). Poiché la troponimia è più difficile da organizzare in un unico albero rispetto all’iponimia dei nomi (non tutti i verbi si raggruppano sotto un’unica radice), i verbi sono rappresentati con una serie di alberi indipendenti, le cui gerarchie tendono comunque ad avere meno livelli, ma più ricchi, di quelle dei nomi.

3.2 Conceptual similarity e misure di similarità in WordNet

Il task di **conceptual similarity** prende in input due termini e restituisce un punteggio numerico che ne indica la vicinanza semantica (per esempio, la similarità fra “car” e “bus” potrebbe essere 0.8 su una scala $[0, 1]$, dove 0 indica sensi completamente dissimili e 1 identità di senso). Per risolvere questo task è naturale sfruttare la struttura ad albero di WordNet: più due synset condividono un antenato vicino, più dovrebbero risultare simili.

Le tre misure classiche viste nel corso sono tutte basate sulla profondità dei synset nella gerarchia (indicata con $depth(x)$, la distanza fra la radice di WordNet e il synset x) e sul loro **Lowest Common Subsumer** (LCS, il primo antenato comune):

- **Wu & Palmer:** $cs(s_1, s_2) = \frac{2 \cdot depth(LCS)}{depth(s_1) + depth(s_2)}$, cioè il doppio della profondità dell’antenato comune, normalizzata sulla somma delle profondità dei due synset: quanto più l’antenato comune è “profondo” (specifico) rispetto ai due synset di partenza, tanto più questi risultano simili;
- **Shortest Path:** $simpath(s_1, s_2) = 2 \cdot depthMax - len(s_1, s_2)$, dove $depthMax$ è la profondità massima dell’albero (un valore fisso per una data versione di WordNet) e $len(s_1, s_2)$ è la lunghezza del cammino più breve fra s_1 e s_2 . Se $len = 0$ (stesso synset) si ottiene il valore massimo $2 \cdot depthMax$; se $len = 2 \cdot depthMax$ (cammino massimo possibile) si ottiene il valore minimo 0: i valori ricadono quindi nell’intervallo $[0, 2 \cdot depthMax]$;
- **Leacock & Chodorow:** $sim_{LC}(s_1, s_2) = -\log \frac{len(s_1, s_2)}{2 \cdot depthMax}$. Per evitare $\log(0)$ quando $s_1 = s_2$ (e quindi $len = 0$), in pratica si aggiunge 1 sia a len sia a $2 \cdot depthMax$; i valori ricadono nell’intervallo $(0, \log(2 \cdot depthMax + 1)]$.

Tutte e tre le formule, però, richiedono in input dei *sensi* (synset), mentre nella pratica si vuole calcolare la similarità fra *termini*, che possono essere polisemici. La soluzione standard è calcolare la similarità fra due termini w_1, w_2 come il **massimo** fra tutte le combinazioni possibili di un senso di w_1 e un senso di w_2 : $sim(w_1, w_2) = \max_{c_1 \in \text{sensi}(w_1), c_2 \in \text{sensi}(w_2)} sim(c_1, c_2)$. L’ipotesi implicita è che i due termini fungano da contesto di disambiguazione reciproco: fra tutte le combinazioni di sensi possibili, quella che massimizza la similarità è presumibilmente quella “giusta” nel contesto in cui i due termini sono stati messi a confronto.

Possibile domanda d’esame. Perché per calcolare la similarità fra due *termini* (anziché fra due sensi) si prende il massimo su tutte le coppie di sensi? — Perché le formule di similarità (Wu&Palmer, Shortest Path, Leacock&Chodorow) sono definite su synset, cioè su sensi specifici, mentre l’input reale è costituito da parole potenzialmente polisemiche. Prendendo il massimo fra tutte le combinazioni di sensi dei due termini si assume implicitamente che i due termini si disambiguino a vicenda: la coppia di sensi più simile è verosimilmente quella effettivamente intesa nel contesto che ha motivato il confronto.

3.3 Word Sense Disambiguation

Il **Word Sense Disambiguation** (WSD) è il problema aperto di identificare quale, fra i vari sensi possibili di una parola polisemica, sia quello effettivamente usato in una frase data. La disambiguazione dei sensi è utile come componente per molti altri task: traduzione automatica, question answering, information retrieval, text classification. Nella sua forma più semplice, un

algoritmo di WSD prende in input una parola nel suo contesto insieme a un inventario di sensi possibili, e restituisce in output il senso corretto per quell'occorrenza specifica; una prima distinzione utile è fra il *lexical sample task* (si sceglie un piccolo insieme predefinito di parole target su cui concentrarsi) e il compito più generale di disambiguare tutte le parole di un testo (*all-words task*).

3.3.1 Feature vector: collocational vs. bag-of-words

Per trasformare il contesto di una parola in input utilizzabile da un classificatore, si esegue prima un minimo di elaborazione linguistica (PoS tagging, lemmatizzazione o stemming, eventualmente parsing sintattico), da cui si estrae un **feature vector** numerico o nominale. Si distinguono due classi di feature: le **feature collocazionali**, che codificano parole (e/o le loro parti del discorso) in posizioni specifiche rispetto alla parola target (esattamente una posizione a destra, esattamente quattro a sinistra, ecc.), e le feature **bag-of-words**, che ignorano la posizione e si limitano a registrare quali parole del vocabolario occorrono da qualche parte nella finestra di contesto.

Un esempio canonico riguarda la disambiguazione del termine *bass* (che in inglese può significare sia “basso” come strumento/voce musicale sia un tipo di pesce) nella frase “An electric guitar and bass player stand off to one side. . .”. Una feature collocazionale estratta da una finestra di due parole a sinistra e due a destra, nel formato $[w_{i-2}, POS_{i-2}, w_{i-1}, POS_{i-1}, w_{i+1}, POS_{i+1}, w_{i+2}, POS_{i+2}]$, darebbe il vettore [guitar, NN, and, CC, player, NN, stand, VB]: il contesto immediato (“guitar”, “player”) suggerisce fortemente il senso musicale. Una feature bag-of-words, costruita invece sulle 12 parole di contenuto più frequenti osservate in un corpus di frasi contenenti “bass” (per esempio [fishing, big, sound, player, fly, rod, pound, double, runs, playing, guitar, band]), rappresenterebbe la stessa frase con un vettore binario che segnala solo la presenza/assenza di ciascuna di quelle parole nella finestra di contesto, senza tenerne traccia della posizione.

3.3.2 L'algoritmo di Lesk

L'algoritmo più studiato per la disambiguazione basata su dizionario è l'**algoritmo di Lesk** (Lesk, 1986), che sfrutta il contesto della parola da disambiguare (le parole a essa vicine) e ne calcola la sovrapposizione (*overlap*) con le glosse dei vari sensi candidati. Nella sua versione semplificata, l'algoritmo (*Simplified Lesk*) inizializza il senso migliore al senso più frequente della parola, poi per ciascun senso candidato costruisce una *signature* (l'insieme delle parole nella gloss e negli esempi WordNet di quel senso) e calcola l'overlap fra questa signature e l'insieme delle parole del contesto; il senso con l'overlap massimo viene restituito come risposta.

Un esempio di manuale: disambiguare la parola *bank* nella frase “the bank can guarantee deposits will eventually cover future tuition costs because it invests in adjustable-rate mortgage securities”, confrontandola con due sensi WordNet: *bank*₁, gloss “a financial institution that accepts deposits and channels the money into lending activities” (con esempi come “he cashed a check at the bank”), e *bank*₂, gloss “sloping land (especially the slope beside a body of water)” (con esempi come “they pulled the canoe up on the bank”). Il contesto della frase (“deposits”, “invests”, “mortgage”, “securities”) condivide molte più parole con la gloss e gli esempi di *bank*₁ che con quelli di *bank*₂: l'algoritmo di Lesk sceglie correttamente il senso finanziario.

Il problema principale dell'algoritmo di Lesk (semplice o originale) è che le voci di dizionario sono spesso troppo brevi per garantire un overlap sufficiente col contesto. Un rimedio, quando sia disponibile un corpus annotato coi sensi, è il **Corpus Lesk**: si arricchisce la signature di ciascun senso aggiungendo tutte le parole osservate nelle frasi del corpus annotate con quel senso, e si pesano le parole in overlap non più per conteggio semplice ma con l'**IDF** (inverse document frequency, la stessa misura standard dell'information retrieval, dove i “documenti” sono qui le glosse e gli esempi): $idf_i = \log \frac{N_{doc}}{n_{d_i}}$, con N_{doc} numero totale di documenti (glosse/esempi) e n_{d_i} numero di quei documenti in cui compare la parola i . Le parole funzionali (“the”, “of”, ecc.), comparando in moltissimi documenti, ottengono così un IDF molto basso, mentre le parole di contenuto ottengono

un IDF alto: il Corpus Lesk usa quindi l'IDF al posto di una lista di stop-word esplicita. Un ulteriore raffinamento è possibile combinando Lesk con un approccio supervisionato, usando le glosse e gli esempi WordNet del senso target per costruire feature bag-of-words supplementari da affiancare (o sostituire) alle parole della frase di contesto.

Un secondo esempio istruttivo riguarda i tre sensi del sostantivo *ash*: (1) il residuo che rimane quando qualcosa viene bruciato; (2) un albero deciduo ornamentale o da legname del genere *Fraxinus*; (3) il legno elastico e resistente ricavato da quegli alberi, usato per mobili, manici di utensili e attrezzi sportivi (per esempio le mazze da baseball). Nel contesto “The house was burnt to ashes while the owner returned” l'overlap lessicale (“burnt”) seleziona chiaramente il senso (1); nel contesto “This table is made of ash wood” l'overlap (“wood”, “table”) seleziona invece il senso (3): lo stesso meccanismo di conteggio della sovrapposizione, applicato a contesti diversi, porta a disambiguazioni diverse e corrette.

Possibile domanda d'esame. Come funziona l'algoritmo di Lesk e quali sono i suoi limiti principali? — Confronta l'insieme di parole del contesto della parola da disambiguare con la signature (parole della glossa e degli esempi) di ciascun senso candidato in WordNet, scegliendo il senso con la sovrapposizione (overlap) maggiore. Il limite principale è che le glosse sono spesso troppo brevi per garantire overlap sufficiente; il Corpus Lesk risolve il problema arricchendo la signature con le parole osservate in un corpus annotato con quel senso, e pesando l'overlap con l'IDF invece che con un conteggio semplice.

3.4 FrameNet

FrameNet parte da un'ipotesi affine a quella già vista per i frame di Minsky: le persone comprendono le cose compiendo operazioni mentali su ciò che già conoscono, e questa conoscenza può essere descritta in “pacchetti di informazione” strutturati. **FrameNet** è un progetto di costruzione di una risorsa lessicale per l'inglese che collega esplicitamente le parole ai loro significati registrando i modi tipici in cui le frasi si costruiscono attorno a esse, sulla base di evidenze raccolte in testi di inglese moderno. I frame funzionano attivando situazioni “stereotipate”: il significato dei singoli concetti dipende dal frame (o dai frame) a cui sono collegati.

Per gestire la polisemia, FrameNet non lavora con semplici parole ma con **unità lessicali** (*lexical unit*, LU): coppie *parola + senso*. Mentre in WordNet sensi diversi della stessa parola appartengono in genere a synset diversi, in FrameNet sensi diversi della stessa parola tendono ad appartenere a frame diversi. L'euristica di lavoro è che se una parola comunica significati differenti in contesti diversi e la differenza non è recuperabile dal solo contesto immediato, è probabile che la parola abbia più di un significato distinto; analogamente, se un verbo dà origine a due costruzioni derivate con pattern sintattici complementari e significati diversi, è il verbo stesso a essere polisemico.

Per descrivere i costituenti di un frame si sviluppa un vocabolario dedicato di **Frame Element** (FE), usato per etichettare le porzioni di una frase che realizzano quel ruolo nel frame. L'esempio guida usato a lezione è il frame REVENGE (vendetta): rappresenta una situazione in cui un individuo *A* ha fatto qualcosa per ferire un individuo *B*; *B* (o un vendicatore che agisce per suo conto) decide di fare qualcosa per ferire *A*; l'azione di ritorsione viene portata a termine indipendentemente da conseguenze legali o istituzionali. Il vocabolario tipico associato al frame comprende nomi (*revenge*, *vengeance*, *reprisal*, *retaliation*, *retribution*), verbi (*avenge*, *revenge*, *retaliate against*, *get back at*, *get even with*, *pay back*), aggettivi (*vengeful*, *vindictive*) e collocazioni verbo+nome (*take revenge*, *exact retribution*, *wreak vengeance*). I Frame Element principali di REVENGE sono l'*Avenger* (chi si vendica), l'*Offender* (il responsabile del torto originario), l'*Injury* (il danno subito) e l'*Injured party* (la parte lesa, che può coincidere o no con l'*Avenger*).

Il lavoro di annotazione in FrameNet distingue due tipi di parole target: i **predicati**, parole che evocano un frame e creano il contesto per le informazioni da inserire (l'obiettivo dell'annotazione,

in questo caso, è trovarne gli argomenti), e i **filler**, parole che soddisfano un ruolo semantico all'interno del frame evocato da un predicato (qui l'obiettivo è identificare a quale frame ed FE appartengano). Una nozione collegata è la **valenza**: il numero e il tipo di argomenti controllati da un predicato, che può essere impersonale, intransitiva, transitiva, ditransitiva o tritransitiva. Parole diverse che evocano lo stesso frame mostrano tipicamente pattern grammaticali diversi per realizzare gli stessi FE: nel frame REVENGE, per esempio, l'Offender può essere realizzato sintatticamente come oggetto diretto, oppure introdotto dalle preposizioni *on*, *against*, *with* o *at* a seconda del verbo scelto.

Oltre alla singola frase, FrameNet è pensato per contribuire alla comprensione del testo (*text understanding*) a un livello più ampio: i dati di FrameNet possono supportare la disambiguazione del senso delle parole, la composizione semantica, la scelta fra analisi sintattiche alternative, l'attivazione di un vocabolario legato a un certo topic, e persino compiti collegati come la risoluzione dell'anafora (pronominale, tramite frasi nominali definite, o tramite quantificatori/ordinali). Il principio di utilizzo generale è che, scelta una parola, si individuano i frame che può evocare, si identificano i ruoli semantici richiesti da ciascun frame, e si cerca di far combaciare questi ruoli con le porzioni della frase in input: il frame che si adatta meglio a tutti i costituenti disponibili è quello scelto come interpretazione. Va notato che FrameNet non fornisce direttamente un algoritmo di disambiguazione: offre piuttosto descrizioni di frame collegati e legami semantici fra le parole, che facilitano il giudizio umano o automatico, e che l'analisi non può in generale procedere parola per parola isolatamente, dato lo stretto legame reciproco fra i costituenti di una frase.

Capitolo 4

Elaborazione del testo: NLTK e spaCy

Prima di poter applicare modelli statistici o vettoriali a un testo, è necessario un livello di elaborazione linguistica di base che trasformi il testo grezzo in unità strutturate su cui poter calcolare. Questo capitolo raccoglie i task fondamentali di preprocessing, così come vengono tipicamente implementati con le librerie Python **NLTK** (Natural Language ToolKit, sia libro sia libreria, con numerosi pacchetti di text processing e dataset) e **spaCy**.

4.1 I task di base

La **tokenizzazione** consiste nel separare una stringa di testo in unità più piccole dotate di significato: la *word tokenization* identifica le singole parole (la più piccola unità di testo con un significato proprio), mentre la *sentence tokenization* suddivide un testo nelle frasi che lo compongono. Si tratta del primo passo di qualunque pipeline di NLP, dato che quasi tutti i task successivi (PoS tagging, parsing, ecc.) operano su sequenze di token.

Il **filtraggio delle stop-word** rimuove le parole che, da sole, non portano significato lessicale pieno ma svolgono una funzione prevalentemente grammaticale (congiunzioni, articoli, preposizioni). È importante notare che questa operazione non è mai neutra: le *context word* (le parole che sopravvivono al filtraggio) forniscono informazioni sui temi dell'articolo e caratterizzano lo stile di scrittura, per cui la scelta di quali parole considerare stop-word va sempre calibrata sul task (lo si è visto concretamente nei laboratori, per esempio dovendo aggiungere alle stop-word standard anche le forme contratte tipiche dei testi informali).

Lo **stemming** riduce le parole alla loro radice (*stem*), il nucleo che resta rimuovendo prefissi e suffissi flessivi/derivativi: per esempio “helping” e “helper” condividono entrambe lo stem “help”. NLTK offre più di uno stemmer, i più diffusi sono il **Porter stemmer** e lo **Snowball stemmer** (un'evoluzione dello stesso approccio, con regole più aggiornate e supporto multilingua); i due possono produrre stem leggermente diversi sulla stessa parola. Lo stemming, essendo un'operazione puramente basata su regole/suffissi, è soggetto a due tipi di errore complementari: l'**understemming** (due parole imparentate che restano ridotte a stem diversi, quando dovrebbero coincidere) e l'**overstemming** (due parole non imparentate che vengono invece ridotte allo stesso stem, generando falsi positivi di somiglianza).

Il **PoS tagging** (Part-of-Speech tagging) etichetta ogni parola con la sua parte del discorso (nome, verbo, aggettivo, determinante, ...), usando un *tagset* di riferimento (tipicamente il Penn Treebank tagset: per esempio DT determinante, NNP nome proprio singolare, VBD verbo al passato, CC congiunzione coordinante, IN preposizione/congiunzione subordinante). Una frase come “The Lusitania had been...” viene quindi annotata token per token, per esempio [(‘The’, ‘DT’), (‘Lusitania’, ‘NNP’), (‘had’, ‘VBD’), ...].

La **lemmatizzazione** riduce le parole alla loro forma canonica da dizionario, il **lemma** (a differenza dello stemming, che si limita a troncare in base a regole morfologiche, la lemmatizzazione punta alla forma che si troverebbe effettivamente cercando la parola su un vocabolario): un

lemma rappresenta un intero gruppo di forme flesse, chiamato *lessema*. La qualità di un lemmatizzatore migliora sensibilmente se gli si fornisce anche l'informazione di PoS tagging della parola da lemmatizzare, perché la stessa forma di superficie può corrispondere a lemmi diversi a seconda della categoria grammaticale (per esempio “leaves” può essere il plurale del nome “leaf” oppure la terza persona singolare del verbo “to leave”).

Infine, la **Named Entity Recognition** (NER) individua nel testo le *named entity*, cioè le frasi nominali che si riferiscono a entità specifiche (persone, organizzazioni, località, date, quantità, ecc.), e ne determina il tipo. Un tipico inventario di categorie comprende ORGANIZATION (es. *Georgia-Pacific Corp.*, *WHO*), PERSON (es. *Eddy Bonte*, *President Obama*), LOCATION (es. *Murray River*, *Mount Everest*), DATE, TIME, MONEY, PERCENT, FACILITY, GPE (*geopolitical entity*, es. *South East Asia*). Applicando il riconoscimento a una frase come “While Ann was coming home, Jess went to Spain with Joy and her new Apple laptop.” un chunker come quello di NLTK etichetta correttamente *Ann*, *Jess* e *Joy* come PERSON e *Spain* come GPE, lasciando fuori “Apple” (nome proprio ma non riconosciuto come organizzazione nel contesto) — un buon esempio di come questi sistemi possano sbagliare silenziosamente, ed è quindi sempre bene ispezionare l'output invece di fidarsene ciecamente.

4.2 spaCy

spaCy è una libreria open source per NLP in Python, pensata per un uso più orientato alla produzione rispetto a NLTK: offre tokenizzazione, PoS tagging, lemmatizzazione, parsing a dipendenze, *sentence boundary detection* (SBD) e NER come funzionalità di base, più alcune funzionalità avanzate — **entity linking** (disambiguare le entità testuali riconducendole a identificatori univoci in una base di conoscenza esterna), **similarity** (confrontare parole, testi o documenti interi), **text classification**, **matching a regole** (trovare sequenze di token sulla base di annotazioni linguistiche, in modo simile alle espressioni regolari ma a livello di token anziché di caratteri), oltre a funzionalità di training e serializzazione dei modelli.

Chiamando l'oggetto `nlp` su un testo, spaCy esegue automaticamente tokenizzazione e tutti gli altri passi configurati nella sua **pipeline**, restituendo in output un oggetto `Doc` arricchito di tutte le annotazioni linguistiche calcolate. Non tutte le funzionalità di spaCy sono indipendenti fra loro: alcune (per esempio il parsing a dipendenze) richiedono che la pipeline sia stata effettivamente caricata e processata sul testo, mentre altre sono disponibili anche senza. Una pipeline addestrata è composta da più componenti, ciascuno dei quali sfrutta modelli statistici appresi su dati etichettati, e sono disponibili modelli per diverse lingue. Quando si processano grandi volumi di testo, questi modelli sono più efficienti se applicati a collezioni di documenti piuttosto che a un documento alla volta (il metodo `nlp.pipe` è pensato esattamente per questo caso d'uso). Per velocizzare l'elaborazione quando non servono tutte le componenti della pipeline, si possono selettivamente **disabilitare** componenti (restano caricati in memoria ma non vengono eseguiti) oppure **escludere** del tutto (non vengono nemmeno caricati).

Capitolo 5

Modelli di Linguaggio e N-grammi

5.1 Perché assegnare una probabilità a una frase

Molti task di NLP possono essere ricondotti, in un modo o nell'altro, al problema di stimare quanto sia *probabile* una sequenza di parole: la traduzione automatica (fra due traduzioni plausibili, scegliere quella più fluente nella lingua di arrivo), la correzione ortografica (fra “about fifteen minuets from” e “about fifteen minutes from”, la seconda è linguisticamente più probabile), lo speech recognition, la summarization, il question answering. Un **modello di linguaggio probabilistico** (*Language Model*, LM) è precisamente un modello che assegna una probabilità a una sequenza di parole.

5.2 N-grammi

Il modello più semplice di questo tipo si basa sugli **n-grammi**: sequenze contigue di n parole (un *bigramma* è una sequenza di due parole, un *trigramma* di tre). Gli n-grammi permettono di stimare la probabilità dell'ultima parola di una sequenza date le parole precedenti, e quindi, applicando la regola a catena della probabilità, la probabilità dell'intera sequenza:

$$P(w_1, \dots, w_N) = P(w_1)P(w_2 | w_1)P(w_3 | w_{1:2}) \cdots P(w_N | w_{1:N-1}) = \prod_{i=1}^N P(w_i | w_{1:i-1}).$$

Il problema pratico di questa formula è che il linguaggio è creativo: una particolare storia $w_{1:i-1}$, per quanto lunga, potrebbe semplicemente non essere mai comparsa nel corpus di addestramento, rendendo impossibile stimarne la probabilità condizionale per conteggio diretto. L'intuizione che risolve il problema è l'**assunzione di Markov**: si approssima la storia intera considerando solo le ultime poche parole, $P(w_i | w_{1:i-1}) \approx P(w_i | w_{i-1})$ nel caso più semplice (modello a **bigrammi**), in cui la probabilità dell'intera sequenza si fattorizza come $P(w_{1:n}) \approx \prod_i P(w_i | w_{i-1})$, e ciascuna probabilità condizionale di bigramma si stima per **Maximum Likelihood Estimation** (MLE): si contano le occorrenze in un corpus e si normalizzano i conteggi in modo che risultino fra 0 e 1, cioè $P(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})}$. Da bigrammi e trigrammi si può naturalmente generalizzare a n-grammi di ordine superiore, al costo di una crescente sparsità dei dati (se ne riparlerà a proposito dello smoothing).

Le probabilità stimate in questo modo colgono anche regolarità puramente sintattiche: per esempio “to” ha un'alta probabilità di essere seguito da un verbo all'infinito, semplicemente perché è così che la lingua è strutturata, non per una qualche conoscenza esplicita di regole grammaticali. Un ultimo accorgimento pratico riguarda il calcolo: poiché le probabilità sono sempre ≤ 1 , moltiplicarne insieme molte fa rapidamente tendere il prodotto a zero, con concreto rischio di *underflow* numerico. Per questo si lavora quasi sempre con **log-probabilità**, sommando i logaritmi anziché moltiplicare le probabilità dirette ($\log(p_1 \cdot p_2) = \log p_1 + \log p_2$): la somma è anche computazionalmente più economica del prodotto.

5.3 Valutare un modello di linguaggio

Un modello di linguaggio si può valutare in due modi complementari. La **valutazione estrinseca** inserisce l'LM dentro un'applicazione reale (traduzione, riconoscimento vocale, ecc.) e osserva come cambiano le performance dell'applicazione al variare del modello. La **valutazione intrinseca** misura invece la qualità del modello indipendentemente da una specifica applicazione a valle: si stimano le probabilità degli n-grammi su un *training set*, e si verifica quanto bene il modello si adatta a un *test set* separato e mai visto in fase di stima.

La misura intrinseca standard è la **perplexity**: dato un modello LM e una sequenza di parole di test $W = \{w_1, \dots, w_N\}$, la probabilità della sequenza secondo il modello è $P_{LM}(W) = \prod_{i=1}^N P_{LM}(w_i | w_{1:i-1})$, la sua log-probabilità media è $\frac{1}{N} \sum_i \log P_{LM}(w_i | w_{1:i-1})$, e la perplexity è definita come

$$\text{PPL}(LM, W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P_{LM}(w_i | w_{1:i-1}) \right).$$

Valori di perplexity più bassi indicano un modello migliore: il modello assegna probabilità più alte alle parole effettivamente osservate nel test set, quindi è meno “sorpreso” (*perplexed*) da esse. Intuitivamente la perplexity può anche essere letta come una misura del *branching factor* medio: quante scelte, in media, il modello considera plausibili a ogni posizione della sequenza.

Possibile domanda d'esame. Cosa misura la perplexity e perché un valore più basso è migliore? — La perplexity è l'esponenziale della log-probabilità media (cambiata di segno) che il modello assegna a una sequenza di test mai vista in training. Un valore basso significa che il modello assegnava, in media, probabilità alte alle parole effettivamente osservate: il modello “prevedeva” correttamente il testo. È una misura di valutazione intrinseca, indipendente da una specifica applicazione a valle, e richiede sempre di separare rigorosamente training set e test set.

5.4 Smoothing

Cosa succede se una parola appartiene al vocabolario ma, nel test set, appare in un contesto (un bigramma) mai osservato durante il training? La stima MLE assegnerebbe a quel bigramma probabilità esattamente zero, il che è indesiderabile: si vuole impedire che parole del vocabolario ottengano zero probabilità solo per un incidente di copertura del corpus di addestramento, altrimenti l'intera sequenza di test risulterebbe impossibile secondo il modello (perplexity infinita) alla prima parola non vista.

La tecnica più semplice è il **Laplace smoothing** (o *add-one*): si aggiunge 1 a ogni conteggio prima di normalizzare in probabilità. Per gli unigrammi, se la stima MLE della probabilità di una parola w è $P(w) = \frac{c_w}{N}$ (con c_w conteggio della parola e N numero totale di token), con Laplace si aggiunge 1 a ogni conteggio e si compensa il denominatore con V (la dimensione del vocabolario): $P_{Lap}(w) = \frac{c_w+1}{N+V}$. Per i bigrammi, la normalizzazione standard $P(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})}$ diventa, con Laplace, $P_{Lap}(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i)+1}{C(w_{i-1})+V}$: questo garantisce che nessun bigramma abbia mai probabilità nulla, anche se non osservato nel training (se per esempio $V = 1446$, ogni conteggio unigramma al denominatore viene aumentato esattamente di 1446).

Lo smoothing si può anche interpretare come una forma di **discounting**: si “sottrae” un po' di massa di probabilità agli eventi osservati per ridistribuirli sugli eventi non osservati. Il *discount relativo* è il rapporto fra il conteggio scontato e quello originale.

Un'alternativa concettualmente diversa allo smoothing per add-one è cambiare *quale* n-gramma si usa in base a quanta evidenza si ha a disposizione. Nel **backoff**, se non si dispone di osservazioni sufficienti per un n-gramma di ordine alto (per esempio nessun esempio del trigramma $w_{n-2}w_{n-1}w_n$), si “retrocede” a un ordine inferiore: si stima $P(w_n | w_{n-1})$ usando il bigramma, e

se anche quello manca si scende fino all'unigramma $P(w_n)$. Nell'**interpolazione**, invece di scegliere un solo ordine, si combinano linearmente le stime di tutti gli ordini contemporaneamente, pesate da coefficienti λ che devono sommare a 1:

$$P(w_n \mid w_{n-2}w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n \mid w_{n-1}) + \lambda_3 P(w_n \mid w_{n-2}w_{n-1}).$$

L'intuizione comune a backoff e interpolazione è che usare meno contesto può aiutare a generalizzare proprio nei contesti sotto-rappresentati nel training set, mentre un n-gramma di ordine più alto, quando i dati per stimarlo sono sufficienti, resta comunque preferibile perché più specifico.

Possibile domanda d'esame. Qual è la differenza fra backoff e interpolazione come strategie di smoothing? — Nel backoff si usa l'n-gramma di ordine più alto per cui esistono osservazioni sufficienti, “retrocedendo” a un ordine inferiore solo quando l'ordine superiore non ha dati (una scelta discreta, in base alla disponibilità di evidenza). Nell'interpolazione si combinano sempre le stime di tutti gli ordini n-gramma contemporaneamente, pesate da coefficienti λ_i che sommano a 1 (una combinazione continua, indipendentemente dalla disponibilità di dati per l'ordine più alto).

Capitolo 6

Vector Space Model, Rocchio e Linear Discriminant Analysis

6.1 Terminologia e term-document matrix

Il **Vector Space Model** (VSM) rappresenta ogni testo come un vettore numerico, così da poter applicare al testo tutto l'armamentario dell'algebra lineare (distanze, prodotti scalari, proiezioni). La terminologia di base è: un **documento** è un'unità di testo indicizzata nel sistema e disponibile per la ricerca; una **collezione** è un insieme di documenti; un **termine** è l'elemento (dalla singola parola a intere locuzioni) che occorre nella collezione; un'**interrogazione** (query) è la richiesta dell'utente, anch'essa espressa come insieme di termini.

L'esempio canonico (Jurafsky) è la **term-document matrix**: le righe corrispondono a termini del vocabolario, le colonne a documenti di una collezione (per esempio quattro opere di Shakespeare), e ogni cella conta quante volte un dato termine occorre in un dato documento. In questo schema, ogni documento è rappresentabile come un vettore *colonna* di conteggi (per esempio, "As You Like It" come $[1, 114, 36, 20]$ per i termini *battle*, *good*, *fool*, *wit*); in una collezione reale i vettori-documento hanno dimensionalità pari a $|V|$, la dimensione del vocabolario, tipicamente dell'ordine delle decine di migliaia. Simmetricamente, si può leggere la matrice per *righe*: ogni termine diventa un vettore le cui componenti sono le sue occorrenze nei vari documenti, ed è esattamente in questa direzione (parole come vettori, anziché documenti come vettori) che si muoveranno più avanti gli word embedding.

L'applicazione classica del VSM è l'**information retrieval**: trovare, in una collezione D di documenti, il documento d che meglio corrisponde a una query q (anch'essa rappresentata come vettore di lunghezza $|V|$). Il presupposto è che documenti simili contengano parole simili, e quindi che la similarità testuale si rifletta in una similarità vettoriale: risolvere il task di IR si riduce quindi a confrontare quanto siano simili due vettori. Poiché questi vettori sono tipicamente molto sparsi (pieni di zeri, dato che ogni documento usa solo una piccola porzione del vocabolario), un dettaglio implementativo rilevante è l'uso di strutture dati efficienti per rappresentazioni sparse.

6.2 Cosine similarity

La metrica più diffusa per confrontare due vettori v e w è la **cosine similarity**, basata sul prodotto scalare $v \cdot w = \sum_{i=1}^N v_i w_i$: il prodotto scalare tende a essere alto quando i due vettori hanno valori alti nelle stesse dimensioni, e vale 0 per vettori ortogonali (dimensioni disgiunte, quindi massima dissimilarità). Il prodotto scalare puro, però, favorisce sistematicamente i vettori più lunghi (un documento più lungo tende ad avere conteggi assoluti più alti anche solo per la sua lunghezza, indipendentemente da quanto sia effettivamente simile al documento con cui lo si confronta), mentre si vuole una metrica indipendente dalla lunghezza. Si introduce quindi la **lunghezza** di un vettore, $|v| = \sqrt{\sum_{i=1}^N v_i^2}$, e si normalizza il prodotto scalare per le lunghezze dei

due vettori. Poiché $v \cdot w = |v||w| \cos \theta$, il prodotto scalare normalizzato è esattamente il coseno dell'angolo θ fra i due vettori:

$$\cos(v, w) = \frac{v \cdot w}{|v||w|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}.$$

Il coseno varia in generale in $[-1, 1]$ (1: stessa direzione; 0: vettori ortogonali; -1: direzioni opposte), ma poiché le frequenze di termini non possono mai essere negative, nel contesto del VSM la cosine similarity resta sempre confinata all'intervallo $[0, 1]$.

6.3 Pesatura dei termini: TF-IDF

Finora si è assunto che il peso di un termine in un documento fosse semplicemente il suo conteggio grezzo. In realtà, per attribuire il giusto peso a un termine, contano due fattori: quanto è frequente *in quel documento*, e quanto è raro *nel resto della collezione*. Il secondo fattore è catturato dall'**Inverse Document Frequency** (IDF): i termini presenti in pochi documenti sono più utili per distinguere quei documenti dal resto della collezione (mentre un termine che occorre in tutti i documenti, come tipicamente le stop-word, non aiuta affatto a discriminare). L'IDF di un termine i è definito come $idf_i = \log \frac{N}{n_i}$, dove N è il numero totale di documenti nella collezione e n_i il numero di documenti in cui il termine i occorre; il logaritmo serve a “schiacciare” il rapporto, dato che N può essere enorme. Combinando la frequenza del termine (TF) con l'IDF si ottiene lo schema **TF-IDF**, $w_{i,j} = tf_{i,j} \cdot idf_i$: il peso del termine i nel documento j premia sia la sua frequenza locale sia la sua rarità globale, penalizzando implicitamente le stop-word senza bisogno di una lista esplicita.

6.4 Il metodo di Rocchio

Alcuni classificatori lineari per la text classification si basano su un **profilo esplicito** della categoria, cioè un documento prototipico che la rappresenta: apprendere un classificatore di questo tipo è spesso preceduto da una riduzione locale dello spazio dei termini, e il profilo di una classe c_i è una lista pesata dei termini la cui presenza (o assenza) è più utile per discriminare quella classe.

Il concetto centrale è quello di **centroide**: dato un insieme X di vettori-termini, il centroide è semplicemente la loro media, $\vec{t}_X := \frac{1}{|X|} \sum_{\vec{t}_d \in X} \vec{t}_d$. Un classificatore costruito con il **metodo di Rocchio** premia due cose contemporaneamente: la vicinanza di un documento di test al centroide degli esempi di training *positivi* per quella classe, e la sua distanza dal centroide degli esempi di training *negativi*. Formalmente, per ogni classe c_i si calcola il peso della k -esima feature come

$$f_k^i = \alpha \frac{1}{|POS_i|} \sum_{d_j \in POS_i} w_{kj} - \beta \frac{1}{|NEG_i|} \sum_{d_j \in NEG_i} w_{kj},$$

dove POS_i e NEG_i sono rispettivamente gli esempi di training etichettati come appartenenti e non appartenenti alla classe c_i , e α, β sono parametri di controllo che regolano l'importanza relativa degli esempi positivi e negativi. Se $\alpha = 1$ e $\beta = 0$, il profilo della classe coincide semplicemente con il centroide dei suoi esempi positivi. Un raffinamento pratico riguarda proprio il ruolo di NEG_i : anziché usare l'intero insieme dei negativi, si può usare un campione mirato dei cosiddetti **near-positives**, cioè i documenti negativi più difficili da distinguere dai positivi (i più “vicini” a essere positivi), sul presupposto che siano proprio questi i casi più informativi per tracciare un confine di decisione accurato.

6.5 Linear Discriminant Analysis

Una volta che un documento è stato immerso nello spazio vettoriale del VSM (una rappresentazione $x_i = (w_{i1}, \dots, w_{ip}) \in \mathbb{R}^p$ su un vocabolario $V = \{t_1, \dots, t_p\}$, tipicamente sparsa), il problema di

classificazione diventa un problema standard di apprendimento supervisionato: da un insieme di coppie di training (x_i, y_i) con $y_i \in \{0, 1\}$, si vuole imparare una regola che assegni un nuovo documento a una delle due classi.

La **Linear Discriminant Analysis** (LDA) affronta questo problema da una prospettiva probabilistica: assume che i vettori di ciascuna classe siano generati da una **distribuzione gaussiana multivariata**, $x \mid y=0 \sim \mathcal{N}(\mu_0, \Sigma)$ e $x \mid y=1 \sim \mathcal{N}(\mu_1, \Sigma)$ — cioè le due classi hanno vettori medi μ_0, μ_1 diversi, ma *condividono la stessa matrice di covarianza* Σ . Il vettore medio di ciascuna classe ne descrive la posizione media nello spazio dei termini; la matrice di covarianza comune descrive invece come le componenti del vettore variano congiuntamente attorno alla propria media di classe. Le probabilità a priori delle classi sono indicate con $P(y=0) = \pi_0$, $P(y=1) = \pi_1$.

Da queste assunzioni, LDA deriva per ciascuna classe k una **funzione discriminante**

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k, \quad k \in \{0, 1\},$$

e assegna il documento x alla classe con punteggio discriminante più alto. Nel caso binario questo si traduce in una regola di decisione lineare della forma $w^T x + b \geq 0$: per questo motivo, benché i dati di partenza siano testuali, il classificatore finale resta *lineare* nella rappresentazione vettoriale prodotta dal VSM, cioè definisce un iperpiano separatore nello spazio dei documenti. Nella funzione discriminante, il termine $x^T \Sigma^{-1} \mu_k$ misura l'allineamento fra il documento x e la media della classe k , tenendo conto della struttura di covarianza; il termine $-\frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k$ è una correzione che dipende solo dal centro della classe, e impedisce che classi con vettori medi semplicemente più “grandi” vengano favorite in modo spurio; il termine $\log \pi_k$ incorpora la probabilità a priori della classe, per cui classi più frequenti nel training risultano leggermente avvantaggiate a parità delle altre condizioni.

In pratica, i parametri $\mu_0, \mu_1, \Sigma, \pi_0, \pi_1$ sono tutti stimati dal training set: le probabilità a priori dalle frequenze relative delle classi, le medie dai vettori-documento medi di ciascuna classe, e la matrice di covarianza comune da una stima “pooled” (aggregata) su entrambe le classi, $\Sigma = \frac{\sum_{x_i \in D_0} (x_i - \mu_0)(x_i - \mu_0)^T + \sum_{x_i \in D_1} (x_i - \mu_1)(x_i - \mu_1)^T}{n_0 + n_1 - 2}$, dove n_0, n_1 sono le numerosità delle due classi. Poiché le rappresentazioni testuali sono spesso molto sparse e ad alta dimensionalità, stimare in modo affidabile un'intera matrice di covarianza $p \times p$ può diventare difficile quando il vocabolario è grande.

Perché la covarianza è così importante, e non basterebbe usare solo la differenza fra le medie? Si consideri un esempio a due sole feature, $x = (x_1, x_2)$, con medie di classe $\mu_0 = (1, 1)$ e $\mu_1 = (3, 2)$: la differenza grezza fra le medie è $(2, 1)$, che suggerirebbe che la feature x_1 sia il doppio più importante di x_2 per discriminare le due classi. Ma se la matrice di covarianza comune è, per esempio, $\Sigma = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$ (cioè x_1 varia molto di più di x_2 all'interno di ciascuna classe), allora $\Sigma^{-1} = \begin{pmatrix} 1/4 & 0 \\ 0 & 1 \end{pmatrix}$, e il vettore dei pesi effettivo diventa $w = \Sigma^{-1}(\mu_1 - \mu_0) = (1/2, 1)$: dopo aver tenuto conto della covarianza, è x_2 a risultare più importante di x_1 , nonostante la differenza grezza fra le medie fosse maggiore su x_1 . La ragione è che x_1 ha una varianza molto più alta all'interno delle classi, e quindi è una feature meno affidabile su cui basare la discriminazione: LDA “pesa” automaticamente meno le feature rumorose, un vantaggio che una semplice differenza di centroidi (come nel metodo di Rocchio) non coglierebbe.

Possibile domanda d'esame. Perché LDA pesa le feature diversamente dalla semplice differenza fra le medie di classe (come farebbe il metodo di Rocchio)? — Perché il discriminante di LDA moltiplica la differenza delle medie per l'inversa della matrice di covarianza comune, $\Sigma^{-1}(\mu_1 - \mu_0)$: una feature con varianza alta all'interno delle classi viene automaticamente ridimensionata (pesata di meno), perché è una feature meno affidabile per discriminare, anche se la differenza grezza fra le medie su quella feature è ampia. Rocchio, basandosi solo sui centroidi, non tiene conto di questa informazione sulla dispersione dei dati.

Capitolo 7

Word Embeddings: Word2Vec e oltre

7.1 Dai vettori sparsi ai vettori densi

Il Vector Space Model del capitolo precedente rappresenta parole e documenti con vettori *lunghi e sparsi*: la loro dimensionalità è pari alla dimensione del vocabolario o della collezione, e la stragrande maggioranza delle componenti è zero. Questo approccio ha limiti pratici evidenti: le feature “a mano” (per esempio, quali informazioni sintattiche includere, come gestire la negazione) devono essere progettate esplicitamente da chi sviluppa il sistema, e la distribuzione delle parole in qualunque lingua naturale è fortemente **zipfiana** (la frequenza della parola all’*i*-esimo posto in una classifica per frequenza è circa $1/i$ volte quella della parola più frequente): la maggior parte delle parole di una lingua è, semplicemente, molto rara.

Questo crea un problema concreto per l’apprendimento automatico: se il training set di un classificatore (per esempio, di sentiment su recensioni) contiene la parola *great* ma mai *fantastic*, un modello addestrato su feature sparse legate alle singole parole non saprà come trattare una recensione che contiene *fantastic*, anche se le due parole sono chiaramente vicine per significato. Il contesto che circonda le due parole, tuttavia, tende a essere simile (se il loro significato è simile, ci aspettiamo che compaiano in enunciati simili): è precisamente questa osservazione, nota come **ipotesi distribuzionale**, il punto di partenza dei **word embedding**. Come sintetizzato da due citazioni classiche, Harris (1954) — “le parole che occorrono in contesti simili tendono ad avere significati simili” — e Firth (1957) — “you shall know a word by the company it keeps” — l’idea è che elementi linguistici con distribuzioni simili abbiano significati simili, e che quindi ogni parola possa essere rappresentata con un vettore che ne descriva il contesto d’uso.

Un primo modo, ancora sparso, di catturare questa idea è il **vettore di co-occorrenza**: per ogni parola w , si contano le co-occorrenze con le altre parole del vocabolario entro una finestra di contesto $[-c, +c]$ attorno a ogni occorrenza di w nel testo (valori negativi indicano parole a sinistra, positivi a destra; c tipicamente varia fra 10 e 20). Per esempio, dalle frasi “a bagel and cream cheese”, “also known as bagel with cream cheese” e “American cuisine the bagel is typically sliced”, il vettore di co-occorrenza di *bagel* mostra un’associazione forte con *cream* e *cheese*, ma anche con parole di contesto più ampio come *cuisine* e *sliced*: un’informazione contestuale già significativa, ottenuta puramente per conteggio. In pratica, per costruire questi vettori si tende a scartare le parole a basso contenuto semantico (determinanti, pronomi, preposizioni) e a normalizzare le forme (per esempio via lemmatizzazione); vettori di questo tipo, costruiti su collezioni ampie come Wikipedia, hanno però dimensione pari all’intero vocabolario, e restano sparsi.

Gli **embedding** risolvono il problema passando a vettori *corti e densi*: dimensionalità d tipicamente fra 50 e 1000 (molto minore di $|V|$), con componenti reali che possono anche essere negative. I vettori densi funzionano meglio dei vettori sparsi in praticamente ogni task di NLP: non se ne comprendono ancora del tutto le ragioni teoriche, ma un’intuizione plausibile è che uno spazio di parametri più piccolo (poche centinaia di dimensioni invece di decine di migliaia) costringa il classificatore a imparare molti meno pesi, aiutando la generalizzazione e riducendo il rischio di overfitting.

7.2 Word2Vec e l'algoritmo Skip-gram

Word2vec ribalta l'ipotesi distribuzionale in un compito di apprendimento concreto: anziché dire che “una parola è caratterizzata dal contesto in cui appare”, il suo obiettivo di training è *predire* il contesto in cui una parola è probabile che occorra — il contesto è definito dalla parola, invece che viceversa. Fra i due algoritmi che compongono il pacchetto software word2vec, quello discusso più in dettaglio è lo **skip-gram**, spesso indicato in modo lievemente improprio con lo stesso nome “word2vec”.

L'intuizione dello skip-gram è quella di trasformare un problema difficile (predire quale parola specifica comparirà vicino a un'altra, su un vocabolario di decine di migliaia di parole) in uno molto più semplice: invece di contare quante volte ciascuna parola occorre vicino, per esempio, ad *apricot*, si addestra un classificatore binario sul compito “è probabile che la parola w compaia vicino ad *apricot*?”. In realtà non interessa la risposta a questa domanda in sé: si tengono invece i pesi appresi dal classificatore come gli embedding delle parole. Il testo naturale fornisce implicitamente i dati di supervisione (**self-supervision**, senza bisogno di alcuna etichettatura manuale): una parola c che occorre vicino alla parola target diventa un esempio positivo (“sì, è un vicino”), mentre parole scelte casualmente al di fuori della finestra di contesto diventano esempi negativi (**negative sampling**). Semplificando ulteriormente, invece di allenare una rete neurale multistrato per predire la parola di contesto più probabile, si allena semplicemente un classificatore di **regressione logistica** a distinguere le due situazioni (vicino / non vicino): un modello molto più semplice ed economico da addestrare.

In sintesi, i passi dell'algoritmo skip-gram (con negative sampling, SGNS) sono: (1) trattare la parola target e ciascuna parola del suo intorno come esempio positivo; (2) campionare casualmente altre parole del lessico, al di fuori dell'intorno, come esempi negativi; (3) usare la regressione logistica per addestrare un classificatore che distingua i due casi; (4) prendere i pesi appresi come gli embedding finali.

7.2.1 Il meccanismo di training

Concretamente, l'addestramento mantiene due matrici distinte, inizializzate a valori casuali: una matrice di **Embedding** (per la parola target) e una matrice di **Context** (per le parole di contesto), ciascuna con una riga per parola del vocabolario e d colonne. A ogni passo si prende una parola target (per esempio *not* nella frase “Thou shalt not make a machine in the likeness of a human mind”), il suo vicino reale (per esempio *thou*, esempio positivo), e alcune parole negative campionate a caso (per esempio *aaron*, *taco*, esempi negativi). Per la parola target si recupera il vettore dalla matrice Embedding, per le parole di contesto (positive e negative) i vettori dalla matrice Context; si calcola il **prodotto scalare** fra l'embedding della parola target e ciascun embedding di contesto, e lo si trasforma in un valore fra 0 e 1 tramite la **funzione logistica** (sigmoide). L'errore si ottiene sottraendo questi punteggi sigmoide dalle etichette obiettivo (1 per i vicini reali, 0 per i negativi), e viene usato per aggiornare i pesi; il ciclo si ripete su tutti gli esempi di training, per un numero fissato di epoche.

Più formalmente, il classificatore skip-gram minimizza una funzione di costo che — se si potesse normalizzare esattamente su tutto il vocabolario M — richiederebbe di calcolare un softmax con M termini al denominatore, computazionalmente proibitivo quando M è dell'ordine dei milioni. L'approssimazione realmente implementata da SGNS sostituisce quel softmax con due termini sigmoidei:

$$C = - \sum_{i=1}^M \left[\sum_{w_j \in \text{ctx}(w_i)} \log \sigma(v_{w_j}^o \cdot v_{w_i}^i) + \sum_{w_j \notin \text{ctx}(w_i)} \log \sigma(-v_{w_j}^o \cdot v_{w_i}^i) \right],$$

dove il primo sigmoide massimizza la vicinanza fra gli embedding delle parole effettivamente in contesto, mentre il secondo (grazie al segno negativo dentro la sigmoide) minimizza la vicinanza rispetto a un piccolo campione di parole non in contesto. In pratica, il numero di esempi negativi

usati per ogni esempio positivo è un iperparametro k (il rapporto fra negativi e positivi): un valore comune è $k = 2$. L'obiettivo di apprendimento, in altre parole, è massimizzare il prodotto scalare fra la parola e i suoi veri vicini, minimizzandolo rispetto alle k parole negative campionate.

Al termine del training, il modello ha imparato due embedding per ogni parola i : quello “target” w_i (dalla matrice Embedding) e quello “di contesto” c_i (dalla matrice Context). La pratica più comune è sommarli, rappresentando la parola i con $w_i + c_i$; un'alternativa è scartare del tutto la matrice Context e usare solo w_i .

7.2.2 Skip-gram vs. CBOW

Lo skip-gram non è l'unica architettura possibile: nel **Continuous Bag-of-Words** (CBOW), la direzione della predizione si inverte, e si cerca di predire la parola centrale a partire dalle parole che la circondano (mentre nello skip-gram si predicono le parole di contesto a partire dalla parola centrale). Nella pratica implementativa (per esempio nella libreria **gensim**), questa scelta è controllata da un solo iperparametro booleano, **sg** (0 = CBOW, 1 = skip-gram): un dettaglio non banale, perché il valore di *default* in **gensim** è CBOW (**sg=0**), non skip-gram — un punto su cui è facile sbagliare se non lo si imposta esplicitamente.

Possibile domanda d'esame. Qual è la differenza fra skip-gram e CBOW, e perché è importante controllare esplicitamente il parametro **sg** quando si usa **gensim**? — Skip-gram usa la parola centrale per predire le parole di contesto; CBOW fa l'opposto, usa le parole di contesto (aggregate, per esempio mediate) per predire la parola centrale. In **gensim.models.Word2Vec**, il parametro **sg** sceglie fra le due architetture (0=CBOW, 1=skip-gram) ma il suo valore di default è CBOW: chi vuole addestrare esplicitamente uno skip-gram (per esempio perché è l'architettura discussa a lezione, o richiesta da una consegna) deve impostare **sg=1** a mano, altrimenti otterrà silenziosamente un modello CBOW.

7.2.3 Iperparametri principali e limiti

Fra i principali iperparametri della classe **Word2Vec** di **gensim**: **vector_size** (dimensionalità dell'embedding, default 100), **window** (ampiezza massima della finestra di contesto, default 5 parole per lato), **epochs** (numero di passate sul corpus), **min_count** (soglia minima di frequenza per includere una parola nel vocabolario), **sample** (soglia di sottocampionamento delle parole molto frequenti, per ridurne il peso sproporzionato), **negative** (numero di esempi negativi per ogni esempio positivo, default 5) e **ns_exponent** (esponente che modella la distribuzione da cui vengono campionati i negativi, tipicamente 0.75, la scelta classica di word2vec). Va inoltre ricordato che il training con più **workers** (thread) paralleli introduce una fonte di non determinismo anche fissando un **seed**, dato che l'ordine di elaborazione fra i thread può variare da un'esecuzione all'altra.

I limiti principali di **word2vec** sono due: la gestione delle parole sconosciute (*out-of-vocabulary*), per cui il modello non dispone di alcun embedding se una parola non compariva nel training, e la sparsità intrinseca di molte parole rare, per cui l'embedding appreso può essere di scarsa qualità. Un'estensione pensata proprio per questo è **fastText**: rappresenta ogni parola come combinazione di sé stessa più un insieme di n -grammi di caratteri che la compongono, apprende uno skip-gram per ciascuno di questi n -grammi, e rappresenta la parola finale come somma di tutti gli embedding dei suoi n -grammi — questo permette di costruire comunque un embedding ragionevole anche per parole mai viste in training, purché condividano sotto-sequenze di caratteri con parole note. Un altro insieme di embedding molto diffuso, con un approccio di apprendimento diverso da SGNS (basato su statistiche globali di co-occorrenza anziché su predizioni locali), è **GloVe** (Global Vectors for Word Representation, Stanford).

7.2.4 Cosa catturano gli embedding: analogie e bias

Un risultato sperimentale che ha reso celebri i word embedding è la loro capacità di catturare **relazioni per analogia** come differenze vettoriali: per esempio, usando embedding GloVe pre-addestrati, “king – man + woman” restituisce come vicino più prossimo il vettore di “queen”, e “berlin – germany + australia” restituisce un vettore vicino a “canberra”. Più in generale, la relazione fra una capitale e il proprio stato (rappresentata come differenza dei due vettori) tende a essere simile indipendentemente dalla coppia paese/capitale scelta.

Questa stessa proprietà, però, ha un rovescio problematico: è stato osservato sperimentalmente che gli algoritmi che imparano rappresentazioni numeriche a partire da corpora reali **amplificano i bias** impliciti nei dati di addestramento. Un esempio spesso citato: la differenza vettoriale fra *father* e *doctor* risulta simile a quella fra *mother* e *nurse*, riflettendo (e rinforzando, se questi embedding vengono poi usati in applicazioni a valle) stereotipi di genere presenti nel testo di addestramento piuttosto che un fatto semantico neutro. È un punto rilevante anche dal punto di vista metodologico: un embedding non è mai una rappresentazione “neutra” del significato, ma il riflesso statistico del corpus (e quindi della società) da cui è stato appreso.

Possibile domanda d’esame. Perché i word embedding possono amplificare bias sociali presenti nei dati, e cosa comporta questo dal punto di vista metodologico? — Gli embedding vengono appresi puramente dalla co-occorrenza statistica delle parole in un corpus reale; se il corpus riflette stereotipi sociali (es. associazioni di genere fra professioni), il modello li internalizza come relazioni geometriche fra vettori (es. $\text{father} - \text{doctor} \approx \text{mother} - \text{nurse}$), e li propaga a qualunque applicazione a valle costruita su quegli embedding. Metodologicamente, questo mostra che un embedding non misura il significato in modo neutro, ma la distribuzione (con tutti i suoi bias) del corpus di addestramento.

Un limite strutturale, comune sia a word2vec sia a GloVe/fastText, resta però il cosiddetto **meaning conflation problem**: questi algoritmi imparano un singolo vettore per ogni forma di parola, che finisce per confondere insieme tutti i sensi distinti di una parola polisemica in un solo embedding medio (l’embedding di “bank” non distingue in alcun modo il senso finanziario da quello di sponda di un fiume). Superare questo limite — costruendo embedding *contestualizzati*, che cambiano a seconda della frase in cui la parola compare — è esattamente il problema che motiva l’introduzione delle reti Transformer, trattate nel prossimo capitolo.

Capitolo 8

Un'introduzione ai Transformer

8.1 Dal meaning conflation agli embedding contestualizzati

Come si è visto, gli algoritmi distribuzionali classici (word2vec, GloVe, fastText) racchiudono tutti i sensi di una parola in un unico embedding, anche quando quei sensi sono chiaramente diversi: è il **meaning conflation problem**. Le reti **Transformer** (*Transformer Networks*, TN) nascono proprio per risolvere questo limite, imparando **contextualized embedding**: rappresentazioni il cui valore cambia a seconda del contesto specifico in cui il termine compare. Concettualmente, ogni parola riceve un embedding ottenuto come combinazione (una media pesata) di alcuni embedding di input, in modo non troppo dissimile da word2vec, ma dipendente dal contesto della frase specifica anziché fisso una volta per tutte.

Una rete Transformer è organizzata come uno **stack di più strati**: l'embedding in uscita dallo strato l diventa l'embedding in ingresso allo strato $l + 1$, tipicamente per un numero di strati che varia fra 2 e 24. Più strati, in generale, permettono alla rete di apprendere funzioni via via più complesse e astratte.

8.2 Architettura

L'input entra nell'**encoder** attraverso uno strato di attenzione e uno strato *Feed-Forward* (FFN); l'output della generazione viene prodotto dal **decoder**, che impiega due strati di attenzione più uno di FFN. Il Transformer originale era organizzato come uno stack di 6 strati sia per l'encoder sia per il decoder, con l'output di ciascuno strato che alimenta il successivo fino alla predizione finale.

L'**encoder** è composto da 6 strati identici, ciascuno formato da due sotto-strati: un meccanismo di *self-attention multi-head*, e una semplice FFN. Attorno a ciascun sotto-strato vengono applicate una **connessione residua** (*residual connection*: si somma l'input originale del sotto-strato al suo output) e una **normalizzazione**; per rendere sempre possibile la connessione residua, ogni sotto-strato e ogni strato producono vettori tutti della stessa dimensione. Il **decoder** è anch'esso composto da 6 strati identici, ma con tre sotto-strati ciascuno: una self-attention multi-head *mascherata* (il mascheramento impedisce di accedere a posizioni future della sequenza, garantendo che la predizione alla posizione i dipenda solo dalle posizioni precedenti), un meccanismo di attenzione sull'output dell'encoder, e una FFN; anche qui, attorno a ogni sotto-strato si applicano connessione residua e normalizzazione.

A seconda di quali componenti vengono effettivamente utilizzate, si distinguono tre grandi famiglie di Transformer: modelli **encoder-only**, che convertono una sequenza di testo in una rappresentazione numerica in cui il vettore calcolato per un dato token dipende sia dal contesto a sinistra sia da quello a destra (*bidirectional attention*) — è la famiglia di BERT e derivati; modelli **decoder-only**, che completano una sequenza data predicendo iterativamente la parola più probabile successiva, guardando solo al contesto a sinistra (*autoregressive* o *causal attention*) — è la

famiglia dei modelli GPT; modelli **encoder-decoder**, usati per modellare mapping complessi fra sequenze di input e output di natura diversa, tipicamente traduzione automatica e summarization — per esempio BART.

Possibile domanda d'esame. Che differenza c'è, in termini di attenzione, fra un modello encoder-only come BERT e uno decoder-only come GPT? — BERT (encoder-only) usa attenzione bidirezionale: la rappresentazione di ogni token dipende sia dal contesto a sinistra sia da quello a destra, il che lo rende adatto a compiti di comprensione/classificazione ma non a generare testo in modo naturale token per token. GPT (decoder-only) usa attenzione autoregressiva/causale, mascherata in modo che ogni token possa vedere solo i token precedenti: questo lo rende adatto alla generazione di testo, predicendo iterativamente la parola successiva.

8.3 I componenti di ogni strato

Ogni strato del Transformer implementa in sequenza: l'aggiunta di informazione posizionale all'embedding di input (l'embedding di ogni parola viene modificato in base alla sua posizione nel testo); la **self-attention** (si generano embedding che sono medie pesate degli embedding dello stadio precedente); somma e normalizzazione.

Il sotto-strato **Feed-Forward** (FFN) viene applicato dopo i meccanismi di attenzione, e ha due scopi principali: permettere una trasformazione non lineare delle rappresentazioni (introducendo complessità che la sola attenzione, lineare nelle sue combinazioni pesate, non potrebbe catturare da sola), e aiutare la rete ad apprendere caratteristiche più astratte (ruoli sintattici, classi semantiche, categorie latenti come tipi di verbi o entità nominate). La FFN si applica in modo indipendente posizione per posizione, ed è composta da due strati lineari separati da una funzione di attivazione non lineare (tipicamente ReLU); apprende tramite backpropagation, ma solitamente senza etichette a livello di singolo token.

La **self-attention** è il meccanismo che permette di associare, all'interno di una frase, “chi sta parlando di cosa”: nell'esempio classico “The animal didn't cross the street because it was too tired”, la self-attention è ciò che permette al modello di collegare correttamente il pronome “it” al suo referente “animal”. Formalmente, per ogni parola i si generano tre vettori tramite proiezioni apprese: una **Query** q_i (rappresenta la parola corrente e “cerca” informazioni rilevanti nelle altre parole), una **Key** k_i (associata a ogni parola, permette di determinare quanto quella parola debba essere presa in considerazione) e un **Value** v_i (contiene l'informazione effettiva che viene “letta” se la parola risulta rilevante). I vettori Query e Key vengono usati per calcolare uno *score* di attenzione fra la parola i e ogni altra parola j ; il punteggio determina quanto i deve prestare attenzione a j , e l'embedding finale di i è una combinazione pesata (secondo questi punteggi) dei vettori Value delle altre parole. Un'analogia utile per l'intuizione: la query è un tipo particolare di pane che si cerca in panetteria; le key sono i nomi di tutti i prodotti disponibili; il value è il prezzo di ciascun prodotto — l'obiettivo è prestare più attenzione (pesare di più) al valore dei prodotti (key) più vicini a ciò che si sta cercando (query).

Il meccanismo di **multi-head attention** ripete questo calcolo in parallelo con più “teste” indipendenti (nel Transformer originale, 8 teste, ciascuna con proiezioni Q, K, V di dimensione $d_k = 64$), permettendo al modello di focalizzarsi contemporaneamente su diverse relazioni/aspetti della sequenza; l'attenzione di ciascuna testa è calcolata come

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V,$$

e dopo il calcolo parallelo delle teste, i risultati vengono concatenati e riproiettati in un unico spazio di dimensione d_{model} (512 nel Transformer originale). Nel decoder, un ulteriore meccanismo di **encoder-decoder attention** permette al decoder di focalizzarsi sulle parti rilevanti dell'input

durante la generazione: in questo sotto-strato, i vettori Key e Value provengono dall'output dell'encoder, mentre i vettori Query provengono dal livello inferiore del decoder — funziona quindi come un'attenzione multi-head standard, ma “incrociata” fra le due sequenze, permettendo al decoder di accedere direttamente all'informazione contestuale dell'input mentre genera l'output.

Poiché l'architettura Transformer, a differenza di modelli sequenziali come le RNN, elabora l'intera sequenza in parallelo, l'ordine delle parole non è implicito nella rappresentazione e va aggiunto esplicitamente tramite il **positional encoding**: ogni parola t riceve, oltre al suo embedding standard w_t , una rappresentazione di posizione p_t , e l'embedding finale in input è la somma $x_t = w_t + p_t$ (a differenza di approcci precedenti che concatenavano la posizione come vettore separato, qui la posizione è integrata direttamente nel vettore parola). La funzione p_t è definita in modo deterministico (non appresa), tipicamente tramite funzioni sinusoidali che variano in modo univoco per ogni posizione e dimensione:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

dove le dimensioni pari del vettore usano il seno e quelle dispari il coseno. L'uso di funzioni sinusoidali permette al modello di apprendere relazioni fra posizioni tramite semplici operazioni lineari, e di generalizzare a lunghezze di sequenza mai viste durante il training.

Infine, i Transformer non operano su parole intere ma su unità **sub-word**, generate automaticamente con l'algoritmo di **Byte Pair Encoding** (BPE): si parte da un vocabolario che contiene tutti i singoli caratteri (più simboli speciali, come un marcatore di fine parola), si scansiona il corpus contando le coppie di simboli adiacenti più frequenti, si sostituisce la coppia più frequente con la sua concatenazione aggiungendola al vocabolario come nuovo simbolo, e si ripete iterativamente il procedimento. Il risultato è un vocabolario di unità sub-word che catturano le sequenze di caratteri più frequenti nel corpus. Tokenizzare a livello di sub-word invece che di parola intera rende il modello più robusto di fronte a parole mai viste (che possono comunque essere ricomposte da sotto-unità già note) e permette di usare un vocabolario complessivo molto più piccolo, con evidenti vantaggi di efficienza.

8.4 Addestramento: pre-training e fine-tuning

L'addestramento di un Transformer consiste nell'ottimizzare i suoi parametri — gli embedding di input dei token sub-word e le matrici di peso W usate nelle proiezioni Q, K, V e nelle FFN dei vari strati — confrontando le predizioni del modello con l'output atteso tramite una **loss function**, che misura quanto le une si discostano dall'altro. Il processo si articola tipicamente in due fasi distinte.

Il **pre-training** è non supervisionato: il modello apprende rappresentazioni linguistiche generali, indipendenti da un'applicazione specifica, da grandi quantità di testo non etichettato. La tecnica più diffusa è il **Masked Language Modeling** (MLM): una frazione dei token in input (tipicamente il 15%) viene sostituita con un token speciale (per esempio [MASK]), e la rete deve indovinare quale token si celi sotto la maschera, sulla base di un classificatore che opera sopra il contextualized embedding prodotto per quella posizione dall'ultimo strato. Il pre-training tramite MLM è non supervisionato nel senso che non richiede annotazioni prodotte da esperti del dominio; poiché il token mascherato può trovarsi in qualunque punto del testo, e il contesto usato per indovinarlo può estendersi sia a sinistra sia a destra della maschera, si parla di **modello di linguaggio bidirezionale** (a differenza dei modelli di linguaggio tradizionali, che procedono sempre da sinistra a destra). Una seconda procedura di pre-training, tipica di BERT, è la **Next Sentence Prediction** (NSP): il modello viene allenato a predire se una data frase segua effettivamente un'altra frase in input, generando esempi positivi da coppie di frasi consecutive del corpus ed esempi negativi da coppie casuali. NSP introduce, oltre a token e positional embedding, un nuovo tipo di embedding che codifica a quale segmento appartenga ciascun token, e il token speciale [CLS], inserito all'inizio dell'input: è trattato come un token qualunque nella self-attention

(quindi accumula, tramite l'attenzione, informazione aggregata da tutta la sequenza), e la sua rappresentazione contestualizzata finale viene usata come input per il classificatore che decide se le due frasi si susseguano oppure no.

Il **fine-tuning**, al contrario, è supervisionato: adatta un Transformer già pre-addestrato a un compito specifico di NLP (classificazione, traduzione, question answering...). I testi di input vengono tipicamente preceduti dal token [CLS] e, se necessario, le diverse porzioni di input sono separate dal token [SEP]; a partire dal modello di base già pre-addestrato (con MLM, NSP, o entrambi), si prosegue l'addestramento con una funzione di loss specifica per il nuovo task (per esempio cross-entropy per un problema di classificazione).

Possibile domanda d'esame. Che differenza c'è fra pre-training e fine-tuning di un Transformer? — Il pre-training è non supervisionato e mira ad apprendere rappresentazioni linguistiche generali da grandi corpora non etichettati, tipicamente tramite Masked Language Modeling (indovinare token mascherati) ed eventualmente Next Sentence Prediction (predire se due frasi si susseguano). Il fine-tuning è supervisionato: parte dal modello pre-addestrato e continua l'addestramento con una loss specifica per un task applicativo concreto (classificazione, traduzione, Q&A), usando un dataset etichettato per quel task.

8.5 Limiti

Due limiti vale la pena ricordare. Il primo è computazionale: l'algoritmo di attenzione ha complessità **quadratica** nella lunghezza della sequenza (ogni token deve calcolare uno score di attenzione con ogni altro token). Questo non è normalmente un problema grazie al parallelismo delle GPU, ma quando queste non sono disponibili le reti Transformer diventano molto più lente di semplici reti neurali feed-forward o ricorrenti. Il secondo riguarda proprio il positional encoding: nella formulazione classica, gli embedding posizionali codificano posizioni *assolute* dei token, mentre architetture più recenti hanno introdotto schemi di codifica posizionale *relativa*, spesso più efficaci soprattutto su sequenze lunghe.